

SENTARI

An Earnings-Call & Filings Intelligence Platform for Retail Investors

Aspect-based sentiment analysis with grounded multi-agent briefs

Project Report

Final Year Project — Sentiment Analysis (231CAUEC51)

Submitted by

Nishant Vikas Narudkar

Ramrao Adik Institute of Technology

D Y Patil University, Navi Mumbai

September 2026

Source code: github.com/nishnarudkar/Sentari

Abstract

Retail investors who follow fifteen to twenty companies cannot read every earnings-call transcript, 10-K/10-Q filing and news item each quarter, so subtle but important signals — softer guidance language, hedging in the Q&A, litigation risk in a footnote — are routinely missed. **Sentari** is a daily intelligence service that ingests these disclosures, measures sentiment at the level of six financial *aspects* (guidance, margins, demand, litigation, management tone and liquidity) instead of one document-level score, tracks how each aspect moves from quarter to quarter, and produces a written brief in which every sentence is traceable to the source sentence that supports it.

The system comprises an idempotent ingestion pipeline with transcript-aware parsing (speakers, prepared remarks versus Q&A), an aspect-extraction stage combining rules with spaCy dependency parsing, and a *model ladder* of ten sentiment models — VADER, SentiWordNet, Loughran-McDonald, a finance-directional rule model, Naive Bayes, Decision Tree, CNN, BiLSTM, and FinBERT both zero-shot and fine-tuned — all evaluated on one held-out gold set and tracked in MLflow. A LangGraph pipeline of Extractor, Bull, Bear, Skeptic and Judge agents writes the brief; the Skeptic acts as a grounding gate that checks citations, numbers, semantic similarity and entailment, and removes unsupported claims before the Judge sees them. Results are served through a FastAPI service and a Next.js dashboard with drill-down from brief to source sentence, together with a drift monitor and a daily digest.

On a hand-written gold set of 94 aspect-labelled sentences, fine-tuned FinBERT reached a macro-F1 of 0.938 and the finance-directional rule model 0.892, against 0.647 for VADER and 0.490 for SentiWordNet. On 26 "trap" sentences where financial meaning inverts everyday polarity (falling costs are good news), VADER was wrong 61.5% of the time and SentiWordNet 73.1%. In an adversarial grounding audit the Skeptic caught every fabricated number, missing or wrong citation and invented claim; the heuristic Skeptic caught only 22.7% of polarity-flipped claims, which rose to 100% with a transformer NLI model.

These numbers are reported with their limitations: the gold set is small, the training data are template-generated weak labels, and the demonstration corpus is synthetic. The signal-validity study and a human grounding audit are designed and implemented but have not yet been run on real data. The report states throughout which parts are measured, which are built but unverified, and which remain to be done.

Keywords: aspect-based sentiment analysis, financial NLP, Loughran-McDonald, FinBERT, multi-agent LLM systems, grounding, hallucination detection, earnings calls

Table of Contents

1	Introduction	1
1.1	Background and motivation	1
1.2	Problem statement	1
1.3	Objectives.....	1
1.4	Scope.....	2
1.5	Contributions	2
1.6	Organisation of this report.....	2
2	Background and Related Work	3
2.1	Levels of sentiment analysis.....	3
2.2	Lexicon- and knowledge-based methods	3
2.3	Machine-learning and deep-learning methods	3
2.4	Transformers and FinBERT	3
2.5	LLM agents, hallucination and grounding.....	3
2.6	Monitoring deployed models.....	4
3	System Design	5
3.1	Architecture overview.....	5
3.2	End-to-end data flow	6
3.3	Database design	7
3.4	Agent orchestration	8
3.5	Technology stack.....	10
4	Implementation	11
4.1	Ingestion service	11
4.2	Preprocessing.....	12
4.3	Aspect extraction	12
4.4	The model ladder	13
4.5	Scoring and the prepared-versus-Q&A signal.....	14
4.6	Model registry and serving	14
4.7	Agent layer	14
4.8	Drift monitoring	15
4.9	API and delivery	16
4.10	Dashboard.....	16

4.11 Configuration and deployment.....	16
5 User Interface Walkthrough.....	18
5.1 Watchlist	18
5.2 Aspect trajectories and drill-down.....	18
5.3 Grounded brief.....	20
5.4 Drift monitor and model registry.....	23
5.5 Dark mode and small screens	25
6 Evaluation.....	27
6.1 Data and protocol	27
6.2 Model ladder	27
6.3 Lexicon failure analysis.....	29
6.4 Grounding audit	31
6.5 Signal validity	32
6.6 Latency and cost.....	33
6.7 Qualitative findings from the end-to-end run	33
7 Testing.....	35
7.1 Strategy	35
7.2 Continuous integration	35
7.3 Defects found and fixed	36
8 Syllabus Mapping	37
9 Ethical and Legal Considerations	38
10 Project Status Against the Plan	39
10.1 Remaining work, in priority order.....	39
11 Limitations and Future Work	40
11.1 Limitations.....	40
11.2 Future work.....	40
12 Conclusion.....	41
References	42
Appendix A Running the System	44
Appendix B Configuration	45
Appendix C API Reference.....	46
Appendix D Repository Structure.....	47

List of Figures

Figure 3.1: System architecture. Dark arrows follow the processing order; grey arrows are reads and writes to the database; the dashed arrow is the registry selecting the serving model.	6
Figure 3.2: Database schema. Solid edges are foreign keys; dashed edges are logical references stored as values.	7
Figure 3.3: Agent graph. Rejected claims never reach the Judge but remain in the audit trail.	9
Figure 4.1: Command-line analysis of a five-sentence passage (rule-based model): each sentence yields one score per detected aspect, with the matched trigger in brackets.	13
Figure 4.2: Interactive API documentation generated by FastAPI (/docs).	16
Figure 5.1: Watchlist: top aspect moves and per-ticker aspect scores from the latest call.....	18
Figure 5.2: Ticker view for ACMX: six aspect trajectories with prepared-remarks and Q&A lines.	19
Figure 5.3: Drill-down: the management-tone point for the Q2 call opens the sentences, sections, scores and model behind it.....	19
Figure 5.4: Source drawer: the selected sentence highlighted in its surrounding transcript, with the model's verdict.....	20
Figure 5.5: Brief for ACMX: stance, confidence breakdown and a summary in which every claim is cited.	20
Figure 5.6: Verified bull and bear claims grouped by aspect.	21
Figure 5.7: Following a citation from the brief to its source sentence.....	22
Figure 5.8: Agent audit trail: the Skeptic's per-claim checks and NLI probabilities.	22
Figure 5.9: Drift monitor: PSI and confidence shift per ticker and for the whole watchlist, with baseline versus current aspect mix.....	23
Figure 5.10: Model registry: every rung with gold-set metrics and the serving flag.....	24
Figure 5.11: MLflow experiment sentari-absa-ladder with one run per model. (Durations are logging times; training times are in Section 6.2.)	24
Figure 5.12: Ticker view in dark mode (NVLT).	25
Figure 5.13: Watchlist at phone width (390 px).	26
Figure 6.1: Macro-F1 by model, coloured by rung.	28
Figure 6.2: Confusion matrices for the general-purpose lexicon, the best rule model and the best learned model.	28
Figure 6.3: Accuracy against inference cost per sentence on CPU (log scale).	29
Figure 6.4: Error rate on finance-inverted (trap) sentences versus the rest.....	30
Figure 6.5: Output of <code>python -m evaluation.lexicon_failure</code>	31
Figure 6.6: Skeptic catch rate by corruption type, heuristic versus NLI entailment.	32

Figure 7.1: Test run: 58 passed..... 35

List of Tables

Table 1.1: Project goals (from the project specification)	1
Table 3.1: Database tables.....	7
Table 3.2: Technology stack (planned in the specification vs. used).....	10
Table 4.1: Code base by package (lines including comments and blank lines)	11
Table 4.2: Data sources	11
Table 4.3: Transcript parsing output (first sentences of the ACMX Q1 2025 sample call).....	12
Table 4.4: Models on the ladder.....	13
Table 4.5: Skeptic checks, applied to every claim in order.....	15
Table 6.1: Evaluation plan status.....	27
Table 6.2: Model ladder on the 94-sentence gold set (P / R / F1 per class).....	27
Table 6.3: Error rate on trap and non-trap sentences.....	30
Table 6.4: Representative trap sentences mis-scored by VADER.....	30
Table 6.5: Adversarial grounding audit (caught = rejected or flagged; hard reject = rejected)	31
Table 6.6: Measured latency	33
Table 7.1: Automated tests by module (58 in total; one test is parametrised six ways)	35
Table 7.2: Defects found through testing and use	36
Table 8.1: Mapping to the Sentiment Analysis course (231CAUEC51).....	37
Table 10.1: Roadmap from the project specification	39
Table B.1: Configuration variables.....	45
Table C.1: HTTP endpoints.....	46
Table D.1: Repository layout	47

Abbreviations

Term	Meaning
ABSA	Aspect-Based Sentiment Analysis
API	Application Programming Interface
BiLSTM	Bidirectional Long Short-Term Memory network
CNN	Convolutional Neural Network
DT	Decision Tree
DVC	Data Version Control
EDGAR	SEC Electronic Data Gathering, Analysis and Retrieval system
F1 / macro-F1	Harmonic mean of precision and recall / unweighted mean of per-class F1
FinBERT	BERT model further trained on financial text (ProsusAI/finbert)
LLM	Large Language Model
LM	Loughran-McDonald financial sentiment dictionary
MD&A	Management's Discussion and Analysis section of a 10-K/10-Q
NB	Naive Bayes
NLI	Natural Language Inference (entailment / neutral / contradiction)
NSE / BSE	National Stock Exchange / Bombay Stock Exchange of India
PSI	Population Stability Index
Q&A	The unscripted question-and-answer part of an earnings call
SWN	SentiWordNet
TF-IDF	Term Frequency – Inverse Document Frequency
VADER	Valence Aware Dictionary and sEntiment Reasoner

1 Introduction

1.1 Background and motivation

Listed companies publish a steady stream of text: quarterly earnings-call transcripts, annual and quarterly reports (10-K and 10-Q in the United States, results and investor presentations filed with NSE and BSE in India), and a larger volume of news. Much of what moves a company's prospects is expressed in this text before it shows up in the numbers — a guidance range narrowed toward its low end, a CFO who becomes evasive about margins in the Q&A, a new lawsuit described in a risk-factor footnote.

Institutional investors pay analysts and data vendors to monitor this. A retail investor following fifteen to twenty stocks usually cannot: a single earnings call runs to ten thousand words or more, and each company produces several documents per quarter. General-purpose sentiment tools do not close the gap, because they score a whole document as positive or negative and because their vocabularies were built for reviews and social media, where "liability", "decrease" or "cost" carry the opposite implications to the ones they carry in finance.

1.2 Problem statement

Build a daily intelligence service that watches a user's watchlist, extracts sentiment about specific financial aspects rather than whole documents, tracks how that sentiment moves quarter over quarter, and produces a grounded, human-readable brief in which every claim can be traced back to the exact sentence that generated it — while being honest, by measurement, about where the approach works and where it does not.

1.3 Objectives

Table 1.1: Project goals (from the project specification)

#	Goal	Where it is addressed
G1	Aspect-level (not document-level) sentiment extraction from unstructured financial text	Chapter 4.3–4.5
G2	Show empirically where general-purpose lexicons fail on financial language	Chapter 6.3
G3	Multi-agent reasoning (bull / bear / skeptic / judge) that is grounded — no hallucinated claims	Chapters 4.7, 6.4
G4	A real deployed service: scheduled ingestion, persistent storage, drift monitoring, a UI	Chapters 4, 5, 10
G5	An honest evaluation, including where the signal does not predict anything	Chapters 6, 11

1.4 Scope

In scope: US and NSE/BSE-listed companies with public transcripts and filings; the six aspects guidance, margins, demand, litigation, management tone and liquidity; a daily batch pipeline; a web dashboard and an e-mail/Slack digest.

Out of scope: real-time or intraday sentiment, trade execution or portfolio management, and any guarantee that a signal predicts returns. Sentari is a research and decision-support tool; every screen, API response and digest carries a not-investment-advice notice.

1.5 Contributions

- An end-to-end, tested pipeline from raw disclosures to a cited brief, runnable fully offline on a bundled synthetic corpus and configurable for live sources.
- A ten-model sentiment ladder scored on one held-out, aspect-labelled gold set, with per-class precision, recall and F1, latency, and MLflow tracking.
- A lexicon failure analysis built around *trap* sentences, quantifying how general-purpose lexicons invert financial meaning.
- A grounding gate (the Skeptic agent) and an adversarial audit that measures what it catches, showing both its strengths and a specific weakness (polarity flips) and how an NLI model removes it.
- A prepared-remarks-versus-Q&A sentiment delta as a per-call signal, and a statistical harness to test whether it predicts post-earnings drift.

1.6 Organisation of this report

Chapter 2 reviews the relevant methods. Chapter 3 presents the system design and Chapter 4 its implementation. Chapter 5 walks through the user interface with screenshots. Chapter 6 reports the evaluation and Chapter 7 the testing. Chapter 8 maps the work to the course syllabus, Chapter 9 covers ethics, Chapter 10 compares progress with the project plan, Chapter 11 discusses limitations and future work, and Chapter 12 concludes. Appendices give run instructions, configuration, the API reference and the repository layout.

2 Background and Related Work

2.1 Levels of sentiment analysis

Sentiment can be analysed at document, sentence or aspect level [10]. Document-level scores are too coarse for earnings calls, where a company commonly reports strong demand, weaker margins and a cautious outlook in the same call. Aspect-based sentiment analysis (ABSA) identifies the targets being evaluated (aspects) and the polarity expressed toward each; it was formalised by the SemEval-2014 shared task [9]. Sentari uses a fixed, domain-defined aspect set and treats aspect detection and polarity scoring as separate stages.

2.2 Lexicon- and knowledge-based methods

VADER [2] is a rule-based lexicon tuned for social media, with heuristics for negation, intensifiers and punctuation. **SentiWordNet 3.0** [3] assigns positive and negative scores to WordNet synsets. Both are general-purpose. Loughran and McDonald [1] showed that almost three-quarters of the words that general dictionaries mark as negative are not negative in 10-K filings (for example *tax*, *cost*, *liability*), and built finance-specific word lists — negative, positive, uncertainty and litigious — now standard in financial text analysis. A known limitation of all word-list methods is directionality: in finance the polarity of *decreased* depends on its subject (falling costs are good, falling margins are bad).

2.3 Machine-learning and deep-learning methods

Supervised classifiers over bag-of-words or TF-IDF features — Naive Bayes and decision trees among them — are strong, cheap baselines [15]. Convolutional networks over word embeddings [5] and recurrent networks such as LSTMs [6] learn local n-gram patterns and longer dependencies respectively, and can capture negation and word order that bag-of-words features miss.

2.4 Transformers and FinBERT

BERT [7] pre-trains a Transformer encoder on large corpora and is fine-tuned for downstream tasks. FinBERT [4] further adapts BERT to financial text and was fine-tuned for sentiment on the Financial PhraseBank [11]. For ABSA, Sun et al. [8] showed that feeding the aspect as an auxiliary second sentence lets one encoder handle all aspects; Sentari's transformer rung uses this sentence-pair formulation.

2.5 LLM agents, hallucination and grounding

Large language models can write fluent summaries but are prone to *hallucination* — statements not supported by their inputs [13]. Common mitigations are retrieval (giving the model the evidence), requiring citations, and verifying claims after generation, for instance with natural language inference (NLI) models that classify whether a premise entails, contradicts or is neutral to a hypothesis. Sentari combines all three: agents argue only from retrieved, scored evidence; each claim must cite chunk IDs; and a separate Skeptic stage verifies claims before they can reach the brief.

2.6 Monitoring deployed models

Deployed NLP models degrade when their input distribution shifts. The Population Stability Index (PSI) [14], widely used in credit scoring, measures the shift between two categorical distributions; Cohen's d [17] measures a standardised shift in a mean. Sentari applies PSI to the mix of aspects mentioned and d to model confidence, per ticker and week.

3 System Design

3.1 Architecture overview

Sentari is organised as a pipeline of services around a single relational store (Figure 3.1). A scheduler triggers the daily run; the ingestion service fetches, deduplicates and chunks documents; the ABSA service tags aspects and scores them with whichever model the registry marks as serving; the agent layer turns scored evidence into a brief; and the delivery layer exposes everything through an API, a dashboard, a digest and a drift monitor.

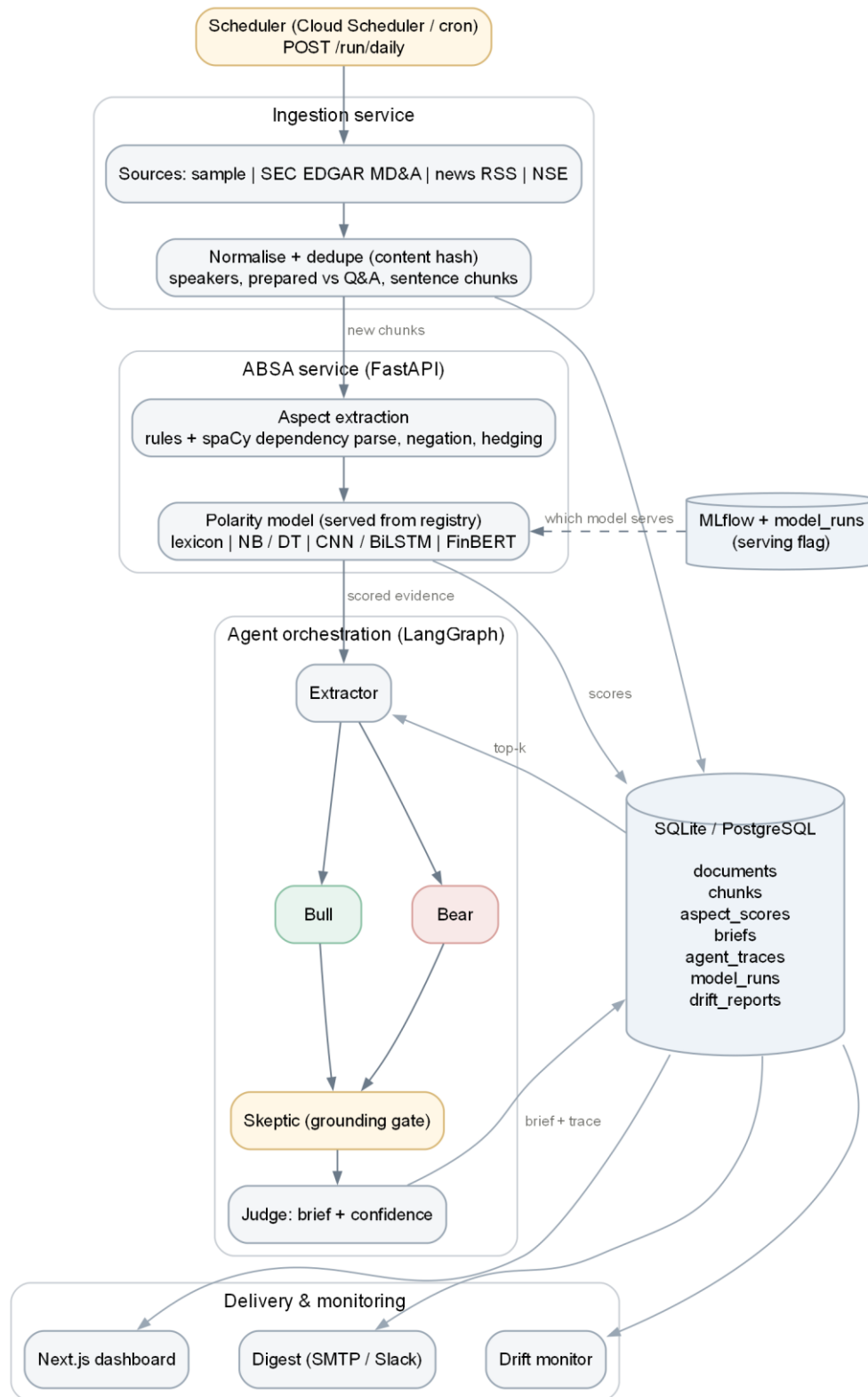


Figure 3.1: System architecture. Dark arrows follow the processing order; grey arrows are reads and writes to the database; the dashed arrow is the registry selecting the serving model.

3.2 End-to-end data flow

1. **Trigger.** Cloud Scheduler (or cron) calls `POST /run/daily`, optionally authenticated with a shared token.

2. **Ingest.** Each configured scraper returns raw documents; text is normalised and hashed; documents already seen are skipped, so re-runs never duplicate data.
3. **Chunk.** Transcripts are split by speaker and section (prepared remarks or Q&A) and then into sentences; filings and news are split into sentences.
4. **Score.** Every new chunk is embedded for retrieval, its aspect mentions are extracted, and each mention is scored by the serving model. Operator lines and analysts' questions are not scored, because they are not company statements.
5. **Brief.** For each ticker whose inputs changed, the agent graph runs and writes the brief and a full audit trail. Unchanged inputs reuse the cached brief.
6. **Monitor and deliver.** Drift reports are computed and the digest of the day's largest aspect moves is rendered and, if configured, sent.

3.3 Database design

All state lives in one SQLAlchemy schema that runs on SQLite for development and tests and on PostgreSQL in production (Figure 3.2, Table 3.1). Aspect scores are denormalised with ticker, date and section so trajectory queries need no joins. Embeddings are stored as JSON arrays so no vector extension is required; the Postgres migration enables pgvector for later ANN search.

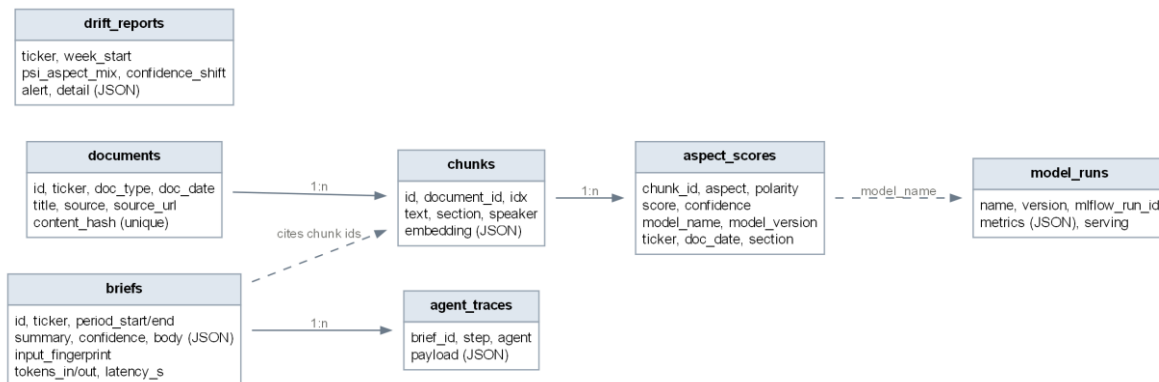


Figure 3.2: Database schema. Solid edges are foreign keys; dashed edges are logical references stored as values.

Table 3.1: Database tables

Table	Purpose	Key constraints
documents	One row per ingested document (transcript, 10-K, 10-Q, news, NSE filing)	<code>content_hash</code> unique — makes ingestion idempotent
chunks	Sentences with section (prepared / qa / mdna / body), speaker and embedding	<code>(document_id, idx)</code> unique
aspect_scores	Polarity, signed score and confidence per chunk × aspect × model	<code>(chunk_id, aspect, model_name)</code> unique
briefs	Final brief as structured JSON, confidence, token counts, latency	<code>input_fingerprint</code> indexed — brief cache
agent_traces	Every agent step of every brief, with cited chunk IDs	ordered by step

Table	Purpose	Key constraints
model_runs	Every trained/evaluated model version, metrics, MLflow run ID	serving flag selects the live model
drift_reports	Weekly PSI and confidence shift per ticker, with alert flag	one row per (ticker, window)

3.4 Agent orchestration

The agent layer is a LangGraph state graph (Figure 3.3). The Bull and Bear agents run in parallel after the Extractor; the Skeptic waits for both; the Judge sees only what the Skeptic lets through. Agent state carries an append-only trace, so every intermediate result is persisted in `agent_traces`.

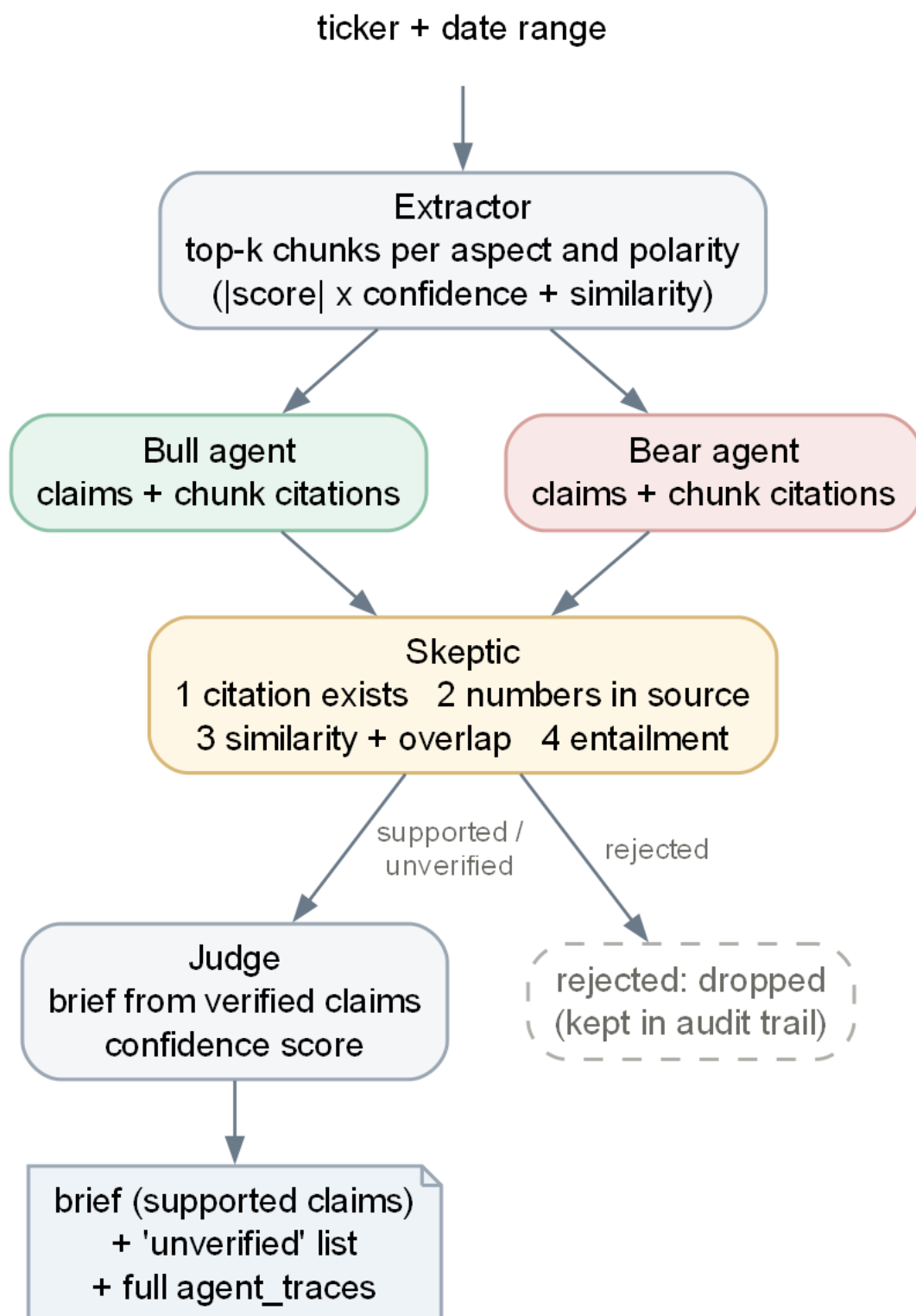


Figure 3.3: Agent graph. Rejected claims never reach the Judge but remain in the audit trail.

3.5 Technology stack

Table 3.2: Technology stack (planned in the specification vs. used)

Layer	Planned	Used in this build
Languages	Python, TypeScript	Python 3.13 (services), TypeScript (dashboard)
NLP	spaCy, NLTK/VADER, SentiWordNet	spaCy en_core_web_sm, vaderSentiment, NLTK SentiWordNet
ML / DL	scikit-learn, PyTorch	scikit-learn 1.8, PyTorch 2.13 (CPU), Hugging Face Transformers
Agents	LangGraph, LangChain	LangGraph; Anthropic SDK (optional LLM backend); LangChain not needed
API	FastAPI	FastAPI + Uvicorn
Tracking / versioning	MLflow, DVC	MLflow (file store or server); dvc .yaml pipeline written
Database / vectors	PostgreSQL, pgvector or Qdrant	SQLite (dev/tests), PostgreSQL (Compose); JSON embeddings
Scheduling	Cloud Scheduler / Airflow	Cloud Scheduler design + /run/daily endpoint; Airflow not used
Containers / deploy	Docker, Compose, Cloud Run	Dockerfiles, Compose and Cloud Run guide written (not yet built or deployed)
Frontend	Next.js	Next.js 16, React 19, dependency-free SVG charts
Notifications	SMTP / Slack API	SMTP and Slack incoming webhook

4 Implementation

The implementation is about 4,000 lines of Python across seven packages and 600 lines of TypeScript, with 58 automated tests (Table 4.1). This chapter describes each component in pipeline order.

Table 4.1: Code base by package (lines including comments and blank lines)

Package	Lines	Responsibility
ingestion/	499	scrapers, normalisation, transcript parsing, idempotent pipeline
absa_service/	2,029	preprocessing, aspect extraction, ten models, training, scoring, signals, registry, API, CLI
agents/	686	extractor, bull/bear, skeptic, judge, LangGraph graph, LLM backends
storage/	180	SQLAlchemy schema, engine/session helpers, migration
monitoring/, delivery/	165	drift monitor; digest rendering, SMTP and Slack delivery
evaluation/	546	metrics, lexicon failure, grounding audit, signal validity, latency/cost, report
tests/	586	58 tests (unit, integration, API)
dashboard/	626	Next.js pages, chart and drawer components, API client

4.1 Ingestion service

Each source implements a `Scraper.fetch(ticker, since)` interface and returns raw documents tagged with ticker, document type and date (Table 4.2). Network scrapers share a client that caches every raw response on disk *before* parsing — so a parsing bug never forces a re-download — and retries with exponential back-off on rate limits.

Table 4.2: Data sources

Scraper	Source	Status
sample	Bundled synthetic transcripts and a 10-Q for two fictional companies (ACMX, NVLT)	Used for all tests and the demonstration
sec_edgar	SEC EDGAR submissions API; extracts the MD&A section of 10-K/10-Q filings [16]	Implemented; not yet run against live EDGAR (needs a contact User-Agent)
news_rss	Public news RSS search by company name	Implemented; not verified live
nse	NSE corporate announcements; returns nothing rather than circumvent access controls	Implemented; not verified live

Normalisation and deduplication. Text is Unicode-normalised, quotes and dashes unified, and stage directions such as *[Operator Instructions]* removed. A SHA-256 hash of ticker, document type and whitespace-normalised text is the document's identity; a unique constraint makes re-ingestion a no-op.

Transcript parsing. Lines beginning with a speaker tag ("Maria Chen -- CEO:") switch the current speaker; phrases such as "question-and-answer session" or the operator opening the line switch the section from prepared remarks to Q&A. Sentences are then split with a regular expression that respects financial abbreviations (*Inc.*, *Q3.*, *U.S.*). Table 4.3 shows the result on the start of a sample call.

Table 4.3: Transcript parsing output (first sentences of the ACMX Q1 2025 sample call)

#	Section	Speaker	Sentence
2	prepared	Maria Chen (CEO)	We delivered a strong first quarter.
5	prepared	Maria Chen (CEO)	We are raising our full-year guidance and now expect revenue growth of 10% to 12%.
7	prepared	David Osei (CFO)	Gross margin expanded 180 basis points to 34.2% on favorable mix and productivity gains.
12	qa	Operator	We will now begin the question-and-answer session.
17	qa	Maria Chen (CEO)	Well, we believe demand should remain healthy, although visibility beyond the third quarter is limited.
23	qa	David Osei (CFO)	We cannot comment on the details, but we could face a material charge if the outcome is unfavorable.

4.2 Preprocessing

- **Negation scope.** A negation cue (*not*, *no*, *without*, *cannot*, *unable*, *...n't*) marks the following tokens as negated until a clause boundary or four tokens. Negated tokens are prefixed `NEG_` for the learned models and flip polarity for the lexicons.
- **Hedging.** Modal and epistemic words (*may*, *could*, *believe*, *expect*, *visibility*, *uncertain*) give each clause a hedge score in $[0, 1]$, used by the management-tone aspect.
- **Clause splitting.** Sentences are split at contrast conjunctions and semicolons, so "Demand held up, **but** aerospace orders declined" yields two separately scored clauses.

4.3 Aspect extraction

Aspect detection combines a trigger lexicon with dependency parsing. Each aspect has a list of trigger terms (for margins: *gross margin*, *operating margin*, *input costs*, *pricing*, *tariffs*, ...). In each clause the earliest matching trigger per aspect is taken. spaCy then parses the clause and, starting from the trigger's syntactic head, collects the *opinion words* that govern it — adjectival modifiers, the head verb, complements and negations — walking up to four ancestors to find negation attached to a governing verb (as in "*we do not expect any material impact from ongoing litigation*"). Without spaCy the extractor falls back to a six-token window; a test checks that both paths detect the same aspects.

```

sentari analyze

PS Sentari> python -m absa_service.cli analyze "<earnings text>"
[lm_directional] margins      positive +0.90 (Gross margin) Gross margin expanded 180 basis points on favorable mix.
[lm_directional] demand      neutral +0.00 (Demand) Demand held up in automation, but aerospace orders declined.
[lm_directional] guidance     negative -0.50 (lowering our) We are lowering our full-year guidance.
[lm_directional] liquidity    positive +0.40 (debt) We repaid 200 million dollars of debt and our leverage ratio improved.
[lm_directional] litigation    negative -0.90 (litigation) We are unable to quantify the litigation exposure.
[lm_directional] management_tone negative -1.00 (unable to) We are unable to quantify the litigation exposure.

```

Figure 4.1: Command-line analysis of a five-sentence passage (rule-based model): each sentence yields one score per detected aspect, with the matched trigger in brackets.

Figure 4.1 shows the behaviour on a mixed passage. The second sentence contains one positive and one negative demand clause; the two scores are averaged to neutral, which is a deliberate simplification discussed in Chapter 11. The last sentence is tagged both litigation and management tone, because *unable to* is a tone trigger.

4.4 The model ladder

All ten models implement one interface, `predict(items) → predictions`, where an item is an aspect clause plus its aspect and a prediction is a label (negative/neutral/positive), a signed score in $[-1, 1]$ and a confidence. This makes every rung interchangeable in the scoring pipeline and directly comparable in evaluation (Table 4.4).

Table 4.4: Models on the ladder

Rung	Model	How it scores a clause
Lexicon	VADER	Compound score of the clause; ± 0.05 thresholds
Lexicon	SentiWordNet	Weighted positive–negative score of the first three synsets per word, sign-flipped under negation
Lexicon	Loughran-McDonald	Positive minus negative word counts with negation flips (curated seed subset of the LM categories; the official dictionary can be dropped in)
Lexicon + rules	LM + directional	LM lists plus directionality: <i>up/down</i> verbs flip sign for metrics where higher is worse (costs, debt, churn, litigation expense); event phrases (<i>named as a defendant, undrawn</i>); uncertainty words for tone
Classical ML	Naive Bayes, Decision Tree	TF-IDF over word 1–2-grams of negation-marked tokens with an aspect token prepended
Deep learning	CNN	Kim-style CNN [5]: 100-d embeddings, filter widths $2/3/4 \times 64$, max-pool, dropout 0.4
Deep learning	BiLSTM	Bidirectional LSTM (96 units per direction) with masked mean pooling; optional GloVe initialisation
Transformer	FinBERT zero-shot	ProsusAI/finbert applied as-is to the (clause, "aspect: ...") sentence pair
Transformer	FinBERT fine-tuned	FinBERT fine-tuned on the sentence pair (2 epochs, AdamW, lr $3e-5$, 1,500 examples, CPU)

Training data. Hand-labelling at scale was out of reach, so, as the project plan anticipated, the trainable models learn from *weak labels*: 4,000 template-generated sentences whose labels follow from financial directionality (for example "input costs fell" → positive, "gross margin fell" → negative),

negation, valence words and event phrases, balanced across the six aspects. Any template sentence that coincides with a gold sentence is removed, and 15% is held out for validation. The consequences of this choice for the evaluation are discussed in Section 6.2.

4.5 Scoring and the prepared-versus-Q&A signal

For each new chunk the scorer extracts aspect mentions, scores each clause with the serving model, averages clauses of the same aspect, and writes one `aspect_scores` row per aspect with ticker, date and section. It also stores an embedding of the chunk for retrieval (a deterministic 384-dimensional feature-hashing embedder by default; Sentence-Transformers optionally).

The platform's signature signal compares what management says in the scripted part of a call with what it says unscripted. For a transcript and aspect a :

$$\Delta(a) = \text{mean score of } a \text{ in Q\&A} - \text{mean score of } a \text{ in prepared remarks}$$

and the per-call delta is the mean of $\Delta(a)$ over aspects present in both sections. A strongly negative delta means management sounded markedly weaker when answering analysts than when reading its script. The naive comparator used in the signal-validity study is the plain document-level mean.

4.6 Model registry and serving

Training (`python -m absa_service.train`) fits every trainable model, evaluates every model on the gold set, logs parameters and metrics to MLflow, records a `model_runs` row, and marks the best model by gold macro-F1 as *serving*. The API resolves which model to use in the order: explicit request → `SENTARI_MODEL` → registry serving flag → the rule-based model. If trained weights are missing the service falls back to the rule-based model rather than failing.

4.7 Agent layer

Extractor

For a ticker and date range the Extractor loads all aspect scores and ranks each aspect's chunks by:

$$\text{rank} = |\text{score}| \times (0.5 + \text{confidence}) + 0.3 \times \text{cosine}(\text{chunk}, \text{aspect query})$$

It keeps the top $k = 4$ positive and top 4 negative chunks per aspect, so both sides of the debate get material, and computes an aspect summary and the per-call Q&A deltas.

Bull and Bear

Each side builds the strongest case from evidence of its polarity and must cite chunk IDs for every claim. The offline backend makes at most one claim per aspect and cites each sentence at most once per side: candidates are taken in order of the Extractor's ranking, and a sentence that triggers several aspects is used only for the aspect it expresses most strongly. The Judge also merges claims from the same side that cite exactly the same sentences, which guards against an LLM restating one sentence under several aspects. With an Anthropic API key the claims are written by Claude, prompted to use only the numbered evidence; without one, a deterministic backend composes claims directly from the

strongest chunks. Claim IDs are prefixed `b1` and `br` so the two sides can never collide in the brief or the trace — a bug that the tests caught during development (Section 7.3).

Skeptic — the grounding gate

Table 4.5: Skeptic checks, applied to every claim in order

Check	Test	On failure
1 Citation	Every cited ID exists in the retrieved evidence	rejected
2 Numbers	Every number in the claim appears in the cited text	rejected
3 Similarity	Embedding cosine ≥ 0.12 and content-word overlap $\geq 45\%$ with the cited text	unverified
4 Entailment	Heuristic: claim polarity cues must not contradict the chunk's scored polarity. NLI mode: cross-encoder entailment ≥ 0.5 ; contradiction ≥ 0.6 rejects	rejected (contradiction) / unverified

Only `supported` claims enter the brief body. `Unverified` claims are shown separately and flagged; `rejected` claims are dropped but kept in the audit trail. Near-verbatim quotes are exempt from the heuristic polarity check, because the cue lists cannot see negation (a faithful quote of *"we do not expect any material impact from litigation"* was otherwise rejected — found during development).

Judge

The Judge assembles the brief from surviving claims: a stance from the net aspect sentiment, bull and bear claims grouped by aspect, the largest negative Q&A delta if below -0.3 , and a confidence score. With an LLM the summary is rewritten in prose, and is discarded in favour of the template if it cites any claim that did not survive. Confidence is a transparent heuristic, not a calibrated probability:

$$\text{confidence} = 0.5 \times \text{survival} + 0.3 \times \text{mean evidence confidence} + 0.2 \times \text{aspect coverage}$$

where $\text{survival} = (\text{supported} + 0.4 \times \text{unverified}) / \text{all claims}$.

Cost control

Each brief is fingerprinted by a hash of its exact inputs (ticker, window, every chunk ID and rounded score, and the LLM name). If a brief with the same fingerprint exists it is returned without running the agents, so the LLM is invoked only when new documents actually arrive for a ticker.

4.8 Drift monitoring

For each ticker and for the watchlist as a whole, the monitor compares the last seven days with the preceding 28:

$$PSI = \sum_a (p_a - q_a) \cdot \ln(p_a / q_a) \quad d = (\mu_{\text{current}} - \mu_{\text{baseline}}) / \sigma_{\text{pooled}}$$

where p and q are the current and baseline aspect-mention shares and d is Cohen's d on model confidence. An alert fires when $PSI > 0.25$ or $|d| > 0.8$; windows with fewer than five scores report *insufficient data*. Reports are keyed by (ticker, window) so re-running the daily job replaces rather than duplicates them.

4.9 API and delivery

The FastAPI service exposes analysis endpoints, the dashboard's read API and the daily-run trigger (Figure 4.2; full list in Appendix C). The digest ranks, across the watchlist, the largest changes in a ticker's per-document aspect mean between its two most recent documents, appends drift alerts and the disclaimer, and sends to Slack and/or e-mail when configured.

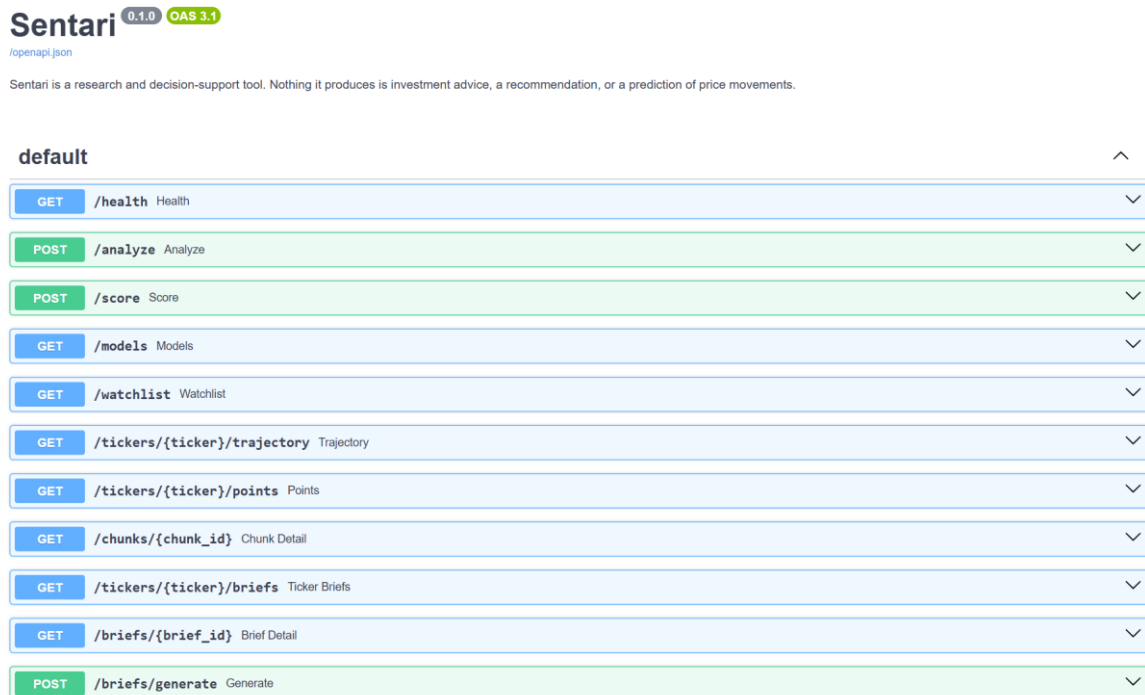


Figure 4.2: Interactive API documentation generated by FastAPI (/docs).

4.10 Dashboard

The Next.js dashboard has five views — watchlist, ticker, brief, drift monitor and model registry — described with screenshots in Chapter 5. Charts are hand-written SVG with no charting dependency; colour tokens adapt to light and dark mode; and every page repeats the research-use disclaimer.

4.11 Configuration and deployment

- **Configuration.** A `.env` file (template: `.env.example`) is loaded before any module reads its settings; real environment variables always win and empty values are ignored. A test fails if the code reads a variable that the template does not document.
- **Containers.** A CPU-only backend image, a standalone Next.js image, and a Compose file adding PostgreSQL (pgvector) and an MLflow server.
- **Cloud.** A Cloud Run guide covers Cloud SQL, both services and a Cloud Scheduler job that calls `/run/daily` with a token; the endpoint is idempotent, so scheduler retries are safe.
- **Reproducibility.** `dvc.yaml` defines the train → lexicon-failure → grounding stages; CI runs the Python tests and the dashboard type-check and build on every push.

Status: the container images have not been built and nothing has been deployed yet (Docker Desktop was not running during development and no cloud project was available). These files are written and reviewed but unverified.

5 User Interface Walkthrough

All screenshots in this chapter were captured from the running system using the bundled synthetic corpus — two fictional companies, Acme Industrial (ACMX) and Novalite Software (NVLT), with two earnings calls each and one 10-Q — after one daily run. The serving model was fine-tuned FinBERT (selected by the registry) and the Skeptic used the NLI backend. Company names, people and figures in these screens are invented.

5.1 Watchlist

The landing page shows the day's largest aspect moves and one card per ticker with the latest call's six aspect scores. A $\triangle$ marks aspects where the Q&A was more than 0.5 below the prepared remarks. The badge shows the latest brief's stance and confidence.

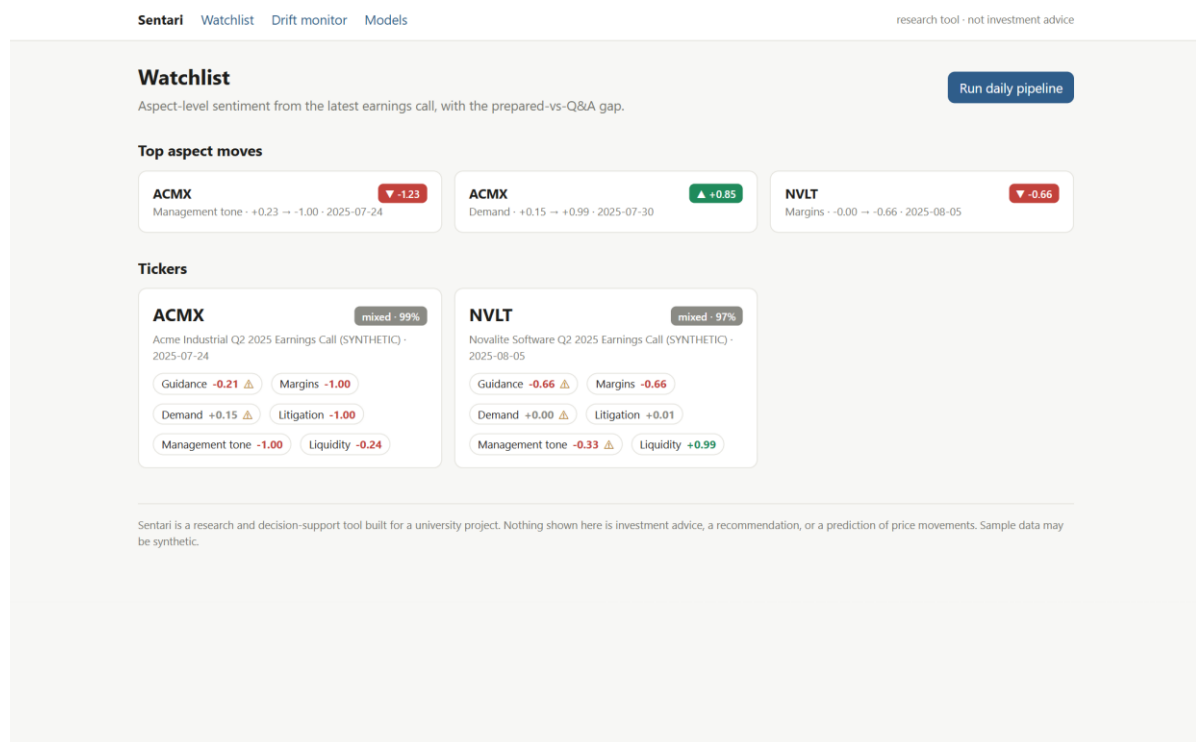


Figure 5.1: Watchlist: top aspect moves and per-ticker aspect scores from the latest call.

5.2 Aspect trajectories and drill-down

The ticker page plots each aspect over time: the solid line is the document mean, the green and red dashed lines the prepared-remarks and Q&A means. For ACMX the Q&A line sits below the prepared line for most aspects — the pattern the prepared-versus-Q&A signal is designed to surface.

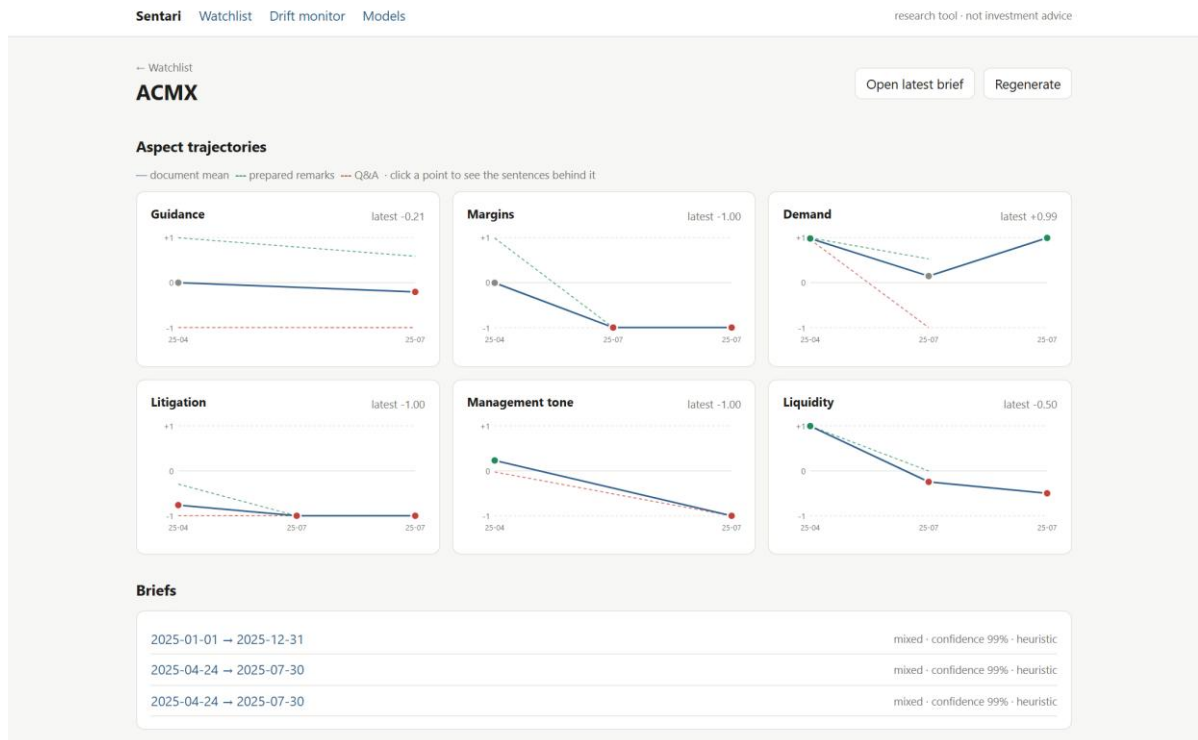


Figure 5.2: Ticker view for ACMX: six aspect trajectories with prepared-remarks and Q&A lines.

Clicking any point lists the scored sentences behind it (Figure 5.3); clicking a sentence opens the source drawer (Figure 5.4), which shows the sentence in context with speakers, the document it came from, and every model verdict stored for it.

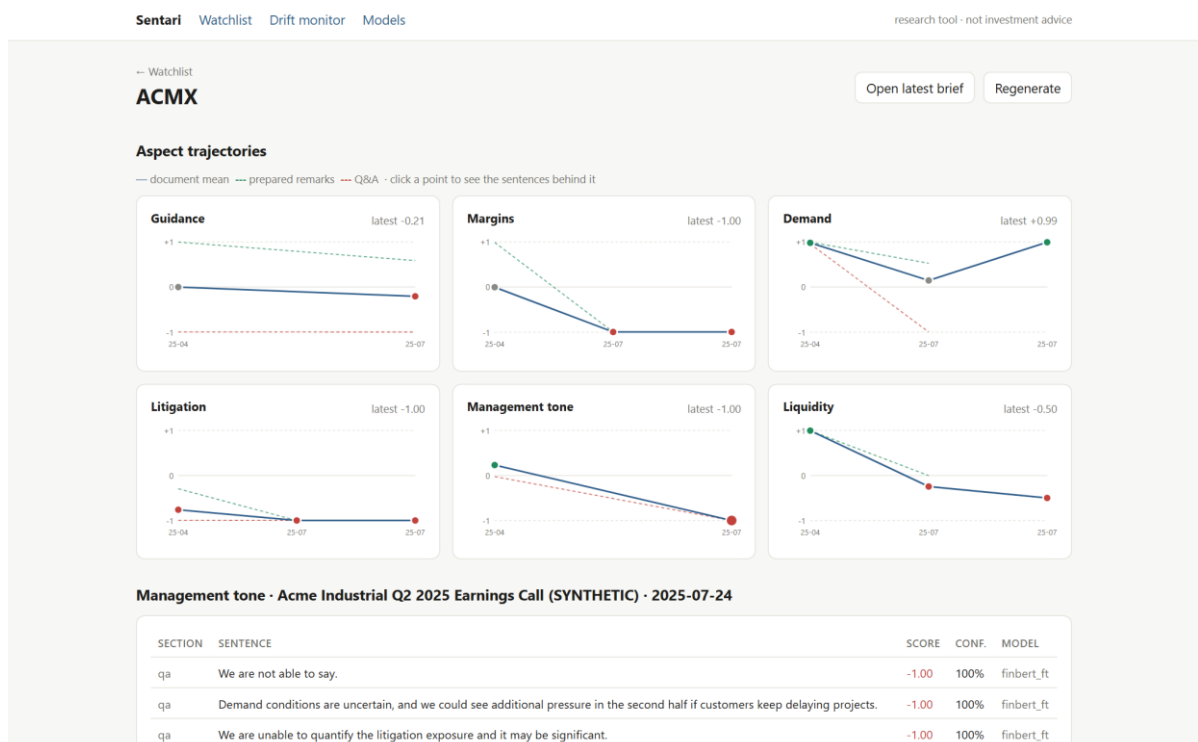


Figure 5.3: Drill-down: the management-tone point for the Q2 call opens the sentences, sections, scores and model behind it.

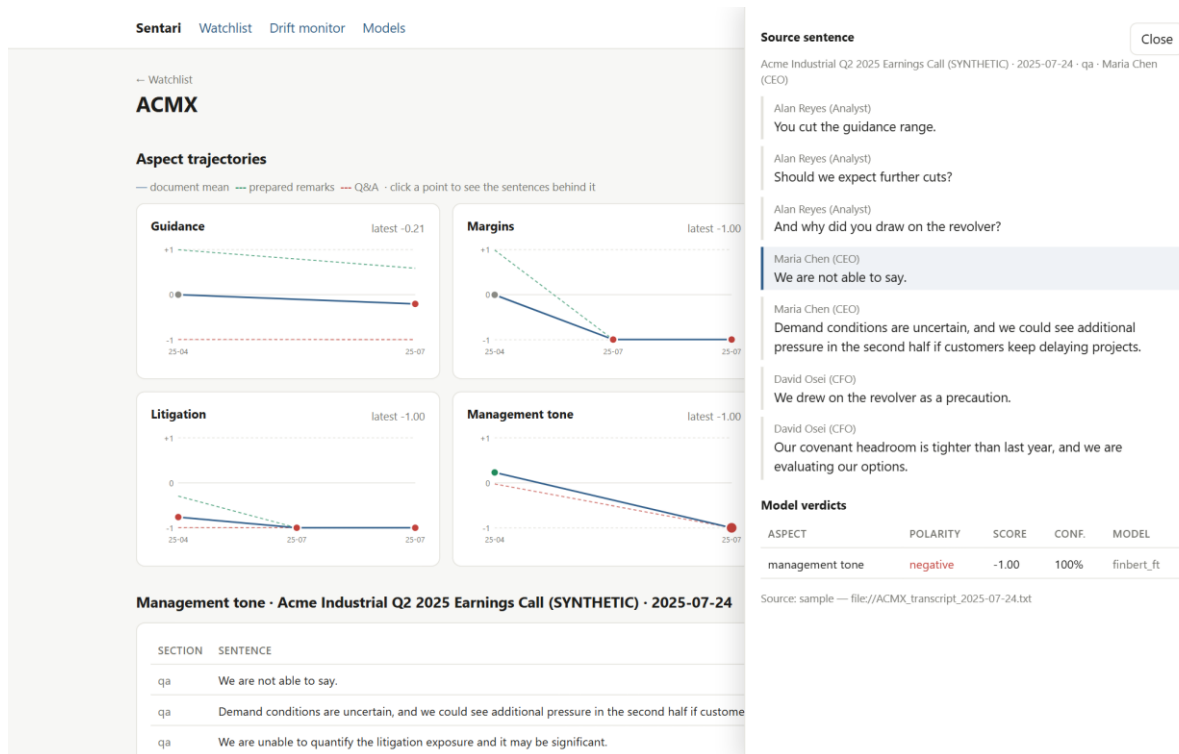


Figure 5.4: Source drawer: the selected sentence highlighted in its surrounding transcript, with the model's verdict.

5.3 Grounded brief

The brief page shows the stance, the confidence with its components, the summary, and the verified bull and bear claims by aspect. Every claim carries *src* chips; each opens the cited source sentence. Unverified claims, if any, are listed separately, and the number of rejected claims is stated.

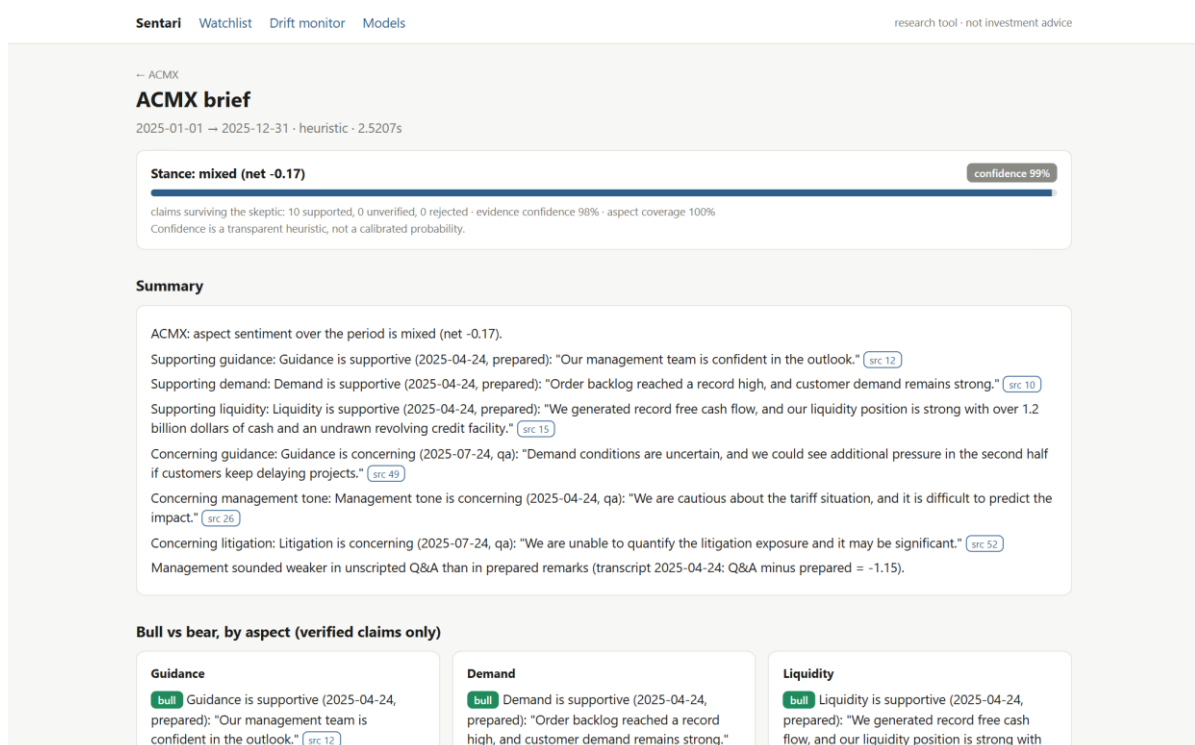


Figure 5.5: Brief for ACMX: stance, confidence breakdown and a summary in which every claim is cited.

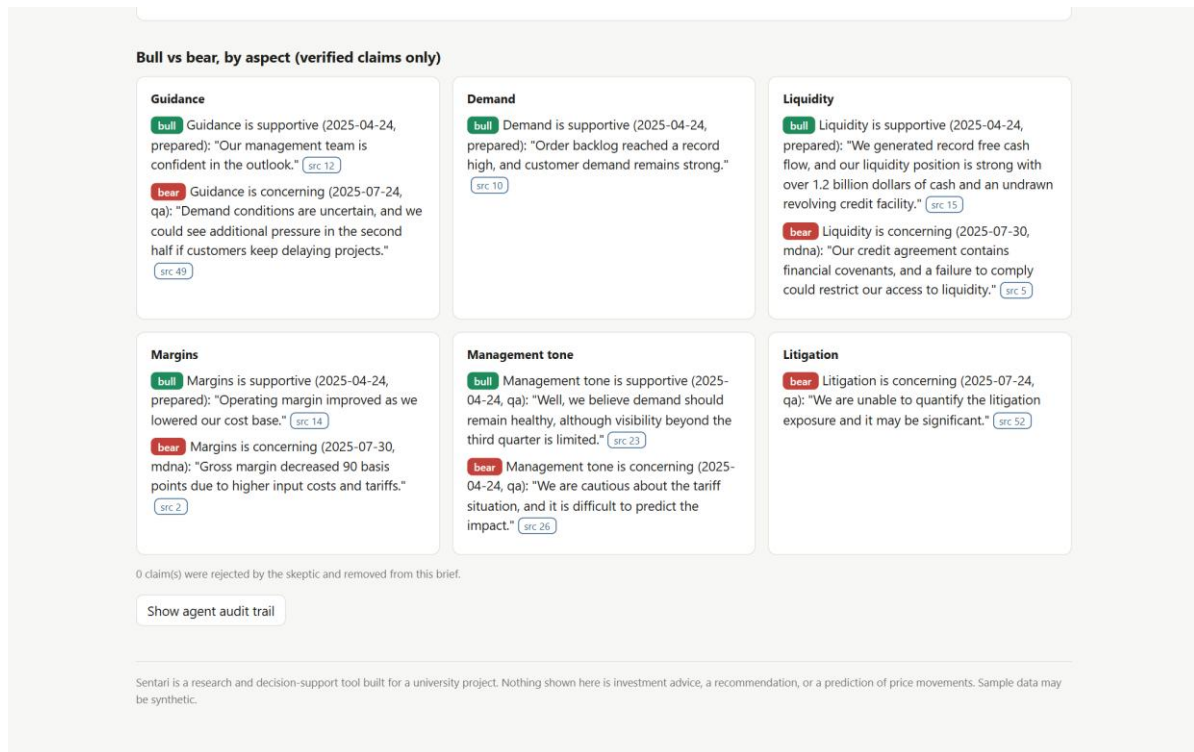


Figure 5.6: Verified bull and bear claims grouped by aspect.

Fixed while preparing this report: repeated claims. The first version of this brief cited one sentence ("Demand conditions are uncertain ...") three times, as the bear claim for guidance, demand *and* management tone, because one sentence can trigger several aspects and each aspect picked its strongest sentence independently. Sentences are now cited at most once per side (Section 4.7). The ACMX brief shown here has 10 claims from 10 different sentences. The bear case lost its demand claim, because no other negative demand sentence exists, and gained a more specific tone claim ("We are cautious about the tariff situation ..."). One imprecision remains: "Our management team is confident in the outlook" is filed under guidance because *outlook* is a guidance trigger.

Sentari
Watchlist
Drift monitor
Models

ACMX
ACMX brief
2025-01-01 → 2025-12-31 · heuristic · 2.5207s

Stance: mixed (net -0.17)

claims surviving the skeptic: 10 supported, 0 unverified, 0 rejected · evidence confidence 98% · aspect coverage 100%

Confidence is a transparent heuristic, not a calibrated probability.

Summary

ACMX: aspect sentiment over the period is mixed (net -0.17).

Supporting guidance: Guidance is supportive (2025-04-24, prepared): "Our management team is confident in the outlook." (src 12)

Supporting demand: Demand is supportive (2025-04-24, prepared): "Order backlog reached a record high, and customer demand remains strong." (src 15)

Supporting liquidity: Liquidity is supportive (2025-04-24, prepared): "We generated record free cash flow of over 1.2 billion dollars of cash and an undrawn revolving credit facility." (src 15)

Concerning guidance: Guidance is concerning (2025-07-24, qa): "Demand conditions are uncertain, and if customers keep delaying projects." (src 49)

Concerning management tone: Management tone is concerning (2025-04-24, qa): "We are cautious about the future." (src 26)

Concerning litigation: Litigation is concerning (2025-07-24, qa): "We are unable to quantify the litigation risk." (src 26)

Management sounded weaker in unscripted Q&A than in prepared remarks (transcript 2025-04-24: Q8)

Bull vs bear, by aspect (verified claims only)

Guidance

bull Guidance is supportive (2025-04-24, prepared): "Our management team is confident in the outlook." (src 12)

Demand

bull Demand is supportive (2025-04-24, prepared): "Order backlog reached a record high, and customer demand remains strong." (src 15)

Source sentence
Close

Acme Industrial Q1 2025 Earnings Call (SYNTHETIC) · 2025-04-24 · prepared · Maria Chen (CEO)

Maria Chen (CEO)

Revenue grew 12% year over year, driven by robust demand across our automation and aerospace segments.

Maria Chen (CEO)

Order backlog reached a record high, and customer demand remains strong.

Maria Chen (CEO)

We are raising our full-year guidance and now expect revenue growth of 10% to 12%.

Maria Chen (CEO)

Our management team is confident in the outlook.

David Osei (CFO)

Gross margin expanded 180 basis points to 34.2% on favorable mix and productivity gains.

David Osei (CFO)

Operating margin improved as we lowered our cost base.

David Osei (CFO)

We generated record free cash flow, and our liquidity position is strong with over 1.2 billion dollars of cash and an undrawn revolving credit facility.

Model verdicts

ASPECT	POLARITY	SCORE	CONF.	MODEL
guidance	positive	+1.00	100%	finbert_ft
management tone	positive	+1.00	100%	finbert_ft

Source: sample — file://ACMX_transcript_2025-04-24.txt

Figure 5.7: Following a citation from the brief to its source sentence.

The audit trail (Figure 5.8) exposes the output of every agent. For the Skeptic it records, per claim, each check's result, the similarity and overlap values and the NLI probabilities — here 0.976 entailment for a quoted claim.

▼ 4. skeptic

```

{
  "agent": "skeptic",
  "verdicts": {
    "supported": 10,
    "unverified": 0,
    "rejected": 0
  },
  "claims": [
    {
      "id": "b11",
      "side": "bull",
      "aspect": "guidance",
      "text": "Guidance is supportive (2025-04-24, prepared): \"Our management team is confident in the outlook.\",",
      "cited_chunk_ids": [
        12
      ],
      "status": "supported",
      "support": {
        "checks": {
          "citation": true,
          "numbers": true,
          "similarity": true,
          "overlap": true,
          "entailment": true
        },
        "similarity": 1,
        "overlap": 1,
        "nli": {
          "entailment": 0.976,
          "neutral": 0.022,
          "contradiction": 0.002
        },
        "reason": "ok"
      }
    },
    {
      "id": "b12",
      "side": "bull",
      "aspect": "demand",
      "text": "Demand is supportive (2025-04-24, prepared): \"Order backlog reached a record high, and customer demand remains strong.\",",
      "cited_chunk_ids": [
        15
      ],
      "status": "supported",
      "support": {
        "checks": {

```

Figure 5.8: Agent audit trail: the Skeptic's per-claim checks and NLI probabilities.

5.4 Drift monitor and model registry

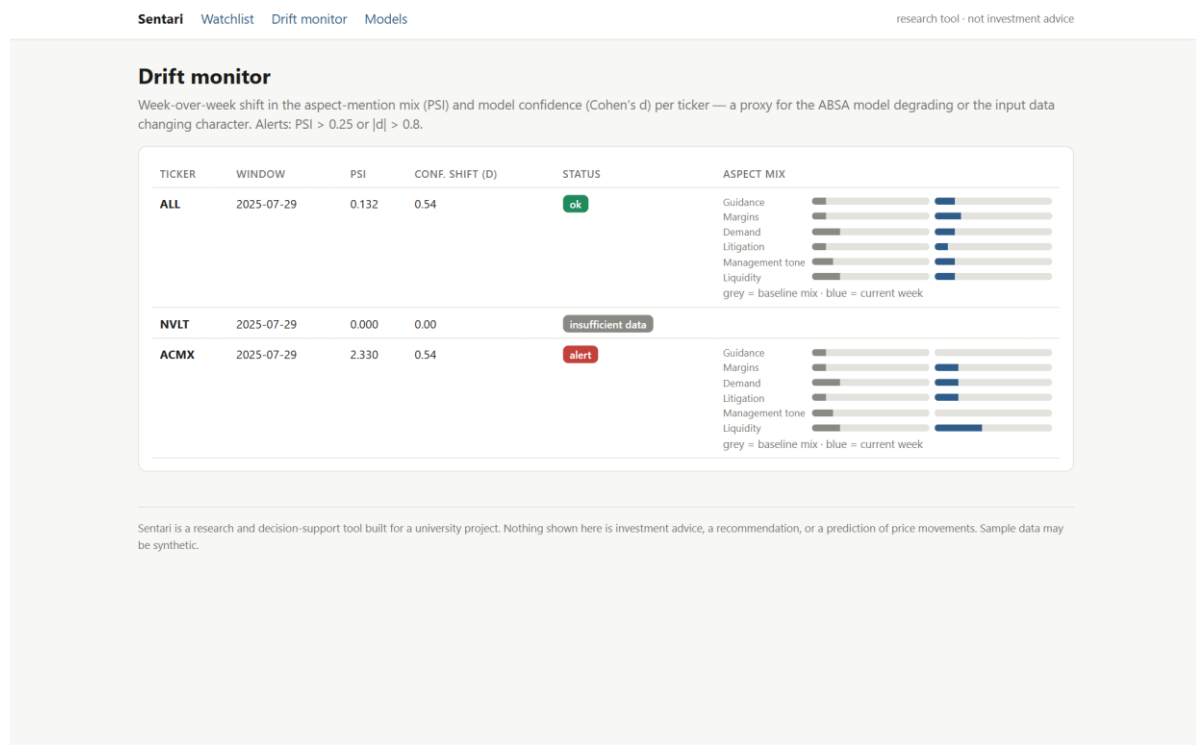


Figure 5.9: Drift monitor: PSI and confidence shift per ticker and for the whole watchlist, with baseline versus current aspect mix.

The ACMX alert (PSI 2.33) illustrates a limitation rather than real drift: the current window contains only five scores, all from one 10-Q, whose aspect mix naturally differs from calls. The five-score minimum is too low for a watchlist this small; Chapter 11 discusses it.

Sentari Watchlist Drift monitor Models research tool - not investment advice

Model registry

Every rung of the ladder, scored on the same held-out gold set. Full runs are tracked in MLflow.

MODEL	VERSION	MACRO-F1	ACCURACY	TRAP ACC.	MS/ITEM	SERVING
finbert_ft	20260921161316	0.938	0.936	0.885	10.475	serving
finbert_zeroshot	20260921160910	0.601	0.606	0.346	8.708	
lstm	20260921160634	0.607	0.638	0.808	0.296	
cnn	20260921160606	0.739	0.745	0.923	0.083	
dt_tfidf	20260921160556	0.519	0.532	0.577	0.038	
nb_tfidf	20260921160556	0.743	0.745	0.692	0.041	
lm_directional	20260921160555	0.892	0.894	0.846	0.034	
lm_lexicon	20260921160555	0.756	0.755	0.423	0.007	
sentiwordnet	20260921160555	0.490	0.489	0.269	1.656	
vader	20260921160555	0.647	0.649	0.385	0.032	

Sentari is a research and decision-support tool built for a university project. Nothing shown here is investment advice, a recommendation, or a prediction of price movements. Sample data may be synthetic.

Figure 5.10: Model registry: every rung with gold-set metrics and the serving flag.

mlflow 3.10.1

GenAI Model training

← sentari-absa-ladder

Overview

Observability

Traces

Sessions

Evaluation

Judges

Datasets

Evaluation runs

Prompts & versions

Prompts

Agent versions

AI Gateway

Assistant Beta

Docs Settings

Experiments > sentari-absa-ladder >

Training runs

metrics.rmse < 1 and params.model = "tree" Time created State: Active Datasets + New run

Sort: Created Columns Group by

Run Name	Created	Dataset	Duration	Source	Models
finbert_ft	3 days ago	-	127ms	C:\Users...	-
finbert_zeroshot	3 days ago	-	126ms	C:\Users...	-
lstm	3 days ago	-	112ms	C:\Users...	-
cnn	3 days ago	-	116ms	C:\Users...	-
dt_tfidf	3 days ago	-	178ms	C:\Users...	-
nb_tfidf	3 days ago	-	199ms	C:\Users...	-
lm_directional	3 days ago	-	83ms	C:\Users...	-
lm_lexicon	3 days ago	-	71ms	C:\Users...	-
sentiwordnet	3 days ago	-	76ms	C:\Users...	-
vader	3 days ago	-	82ms	C:\Users...	-

10 matching runs

Figure 5.11: MLflow experiment sentari-absa-ladder with one run per model. (Durations are logging times; training times are in Section 6.2.)

5.5 Dark mode and small screens

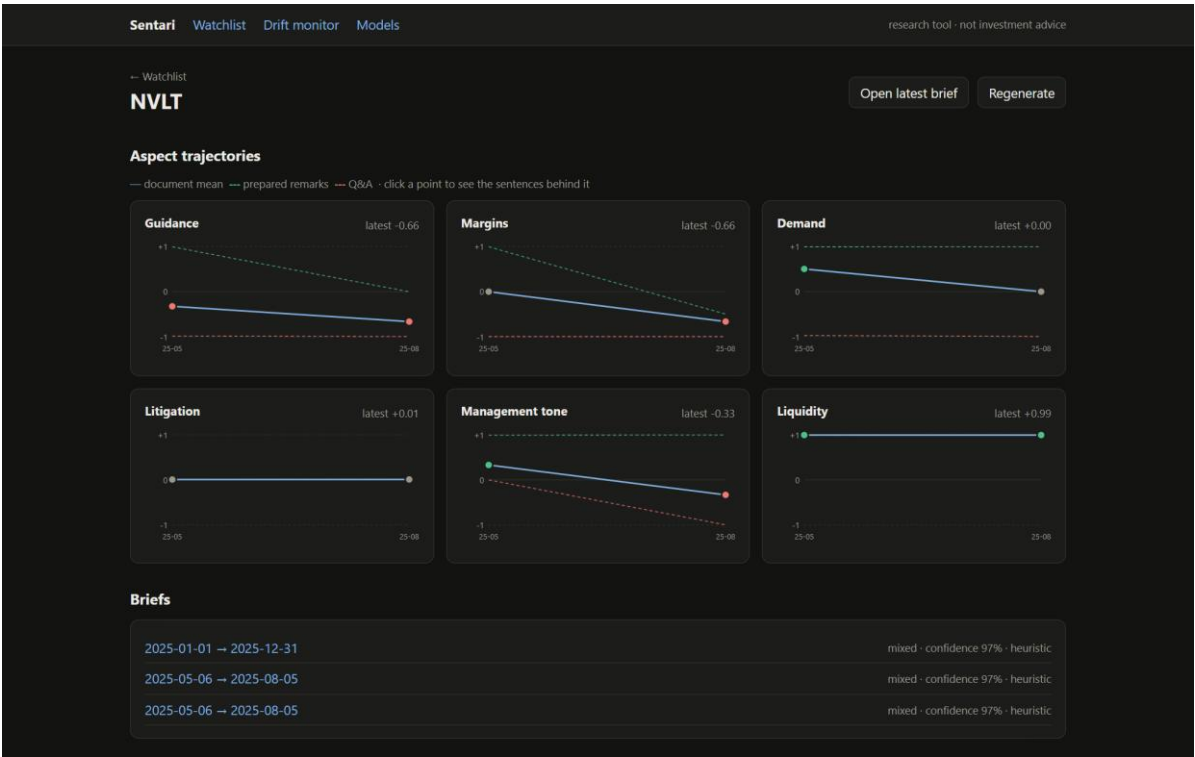


Figure 5.12: Ticker view in dark mode (NVLT).

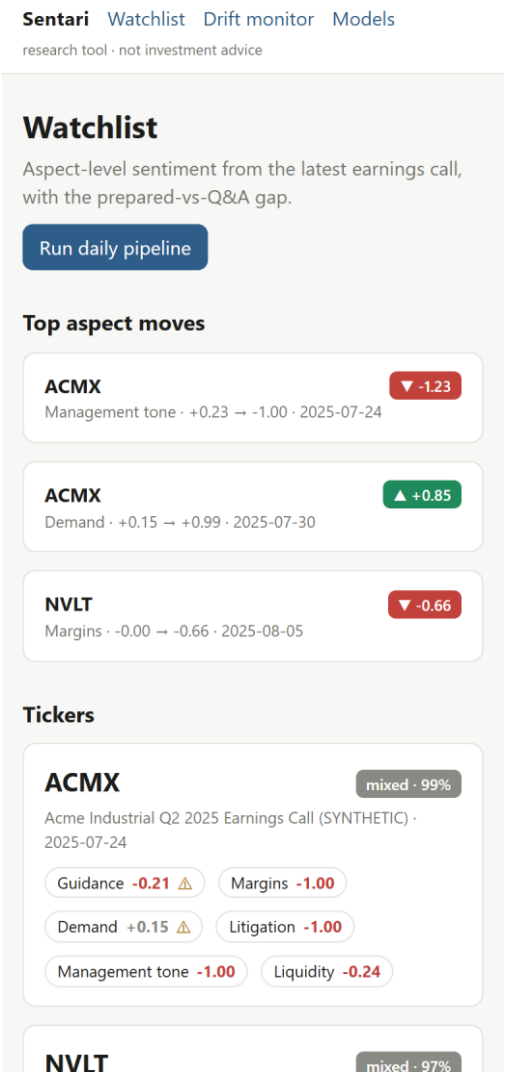


Figure 5.13: Watchlist at phone width (390 px).

At 390 px the cards reflow to a single column. The navigation bar wraps: the links stay on one line and the disclaimer moves to a line of its own. (An earlier version let the disclaimer overflow the right edge of the screen; it was fixed while preparing this report.)

6 Evaluation

The project plan asks for five evaluations. Table 6.1 summarises their status; the rest of the chapter reports each one.

Table 6.1: Evaluation plan status

#	Evaluation	Status
1	Model ladder comparison on one held-out set	Done (Section 6.2)
2	Lexicon failure analysis	Done (Section 6.3)
3	Grounding audit	Adversarial audit done; human-annotated audit tooling built, not yet run (Section 6.4)
4	Signal validity (Q&A delta vs. post-earnings drift)	Harness built and tested; not run — needs real transcripts and prices (Section 6.5)
5	Latency and cost per brief	Latency measured; LLM cost not measured — no API key used (Section 6.6)

6.1 Data and protocol

- **Gold (test) set:** 94 hand-written sentences, each labelled with one aspect and a polarity: every aspect has 5–6 positive, 6 negative and 4 neutral examples. 26 are flagged as *traps* — sentences where financial meaning inverts everyday polarity. The gold set is used only for evaluation.
- **Training set:** 3,400 weak-labelled template sentences, with 600 more held out for validation (Section 4.4). FinBERT was fine-tuned on the first 1,500 for CPU time.
- **Metrics:** accuracy, macro-F1, per-class precision/recall/F1, accuracy on the trap subset, and inference time per sentence on CPU. One training seed.

6.2 Model ladder

Table 6.2: Model ladder on the 94-sentence gold set (P / R / F1 per class)

Model	Acc.	Macro-F1	Negative	Neutral	Positive	Trap acc.	ms/item
VADER	0.649	0.647	0.69 / 0.61 / 0.65	0.59 / 0.67 / 0.63	0.66 / 0.68 / 0.67	0.38	0.03
SentiWordNet	0.489	0.490	0.56 / 0.42 / 0.48	0.38 / 0.63 / 0.47	0.59 / 0.47 / 0.52	0.27	1.66
Loughran-McDonald (seed list)	0.755	0.756	0.92 / 0.67 / 0.77	0.57 / 1.00 / 0.73	0.88 / 0.68 / 0.77	0.42	0.01
LM + directional rules	0.894	0.892	0.97 / 0.86 / 0.91	0.79 / 0.96 / 0.87	0.91 / 0.88 / 0.90	0.85	0.03
Naive Bayes (TF-IDF)	0.745	0.743	0.69 / 0.81 / 0.74	0.88 / 0.63 / 0.73	0.74 / 0.76 / 0.75	0.69	0.04
Decision Tree (TF-IDF)	0.532	0.519	0.47 / 0.75 / 0.57	0.58 / 0.46 / 0.51	0.71 / 0.35 / 0.47	0.58	0.04
CNN (Kim-style)	0.745	0.739	0.69 / 0.86 / 0.77	0.83 / 0.63 / 0.71	0.77 / 0.71 / 0.74	0.92	0.08
BiLSTM	0.638	0.607	0.58 / 0.81 / 0.67	0.80 / 0.33 / 0.47	0.68 / 0.68 / 0.68	0.81	0.30

Model	Acc.	Macro-F1	Negative	Neutral	Positive	Trap acc.	ms/item
FinBERT, zero-shot	0.606	0.601	0.67 / 0.39 / 0.49	0.61 / 0.71 / 0.65	0.58 / 0.76 / 0.66	0.35	8.71
FinBERT, fine-tuned	0.936	0.938	0.94 / 0.92 / 0.93	0.96 / 0.96 / 0.96	0.91 / 0.94 / 0.93	0.88	10.47

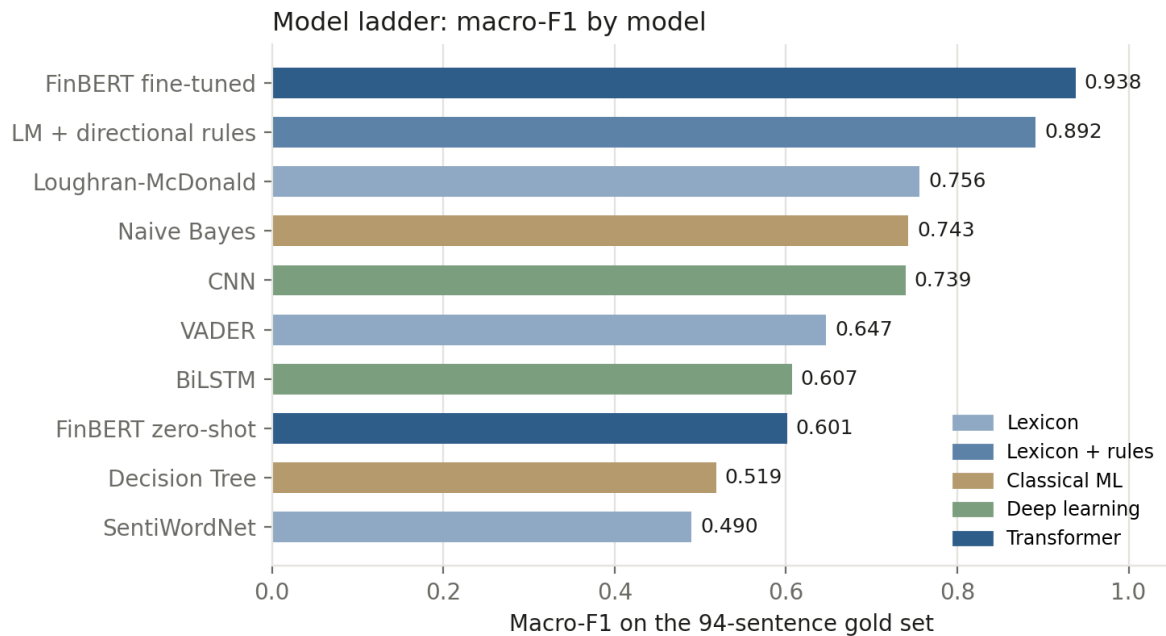


Figure 6.1: Macro-F1 by model, coloured by rung.

Fine-tuned FinBERT is the strongest model (macro-F1 0.938), followed by the LM + directional rule model (0.892). Adding finance-specific vocabulary alone moves the lexicon rung from 0.647 (VADER) to 0.756 (Loughran-McDonald); adding directionality moves it to 0.892. Zero-shot FinBERT (0.601) does worse than plain Loughran-McDonald: it was trained for sentence-level sentiment and does not know which aspect it is being asked about, so fine-tuning on the aspect pair is what makes it useful here.

Confusion matrices on the gold set

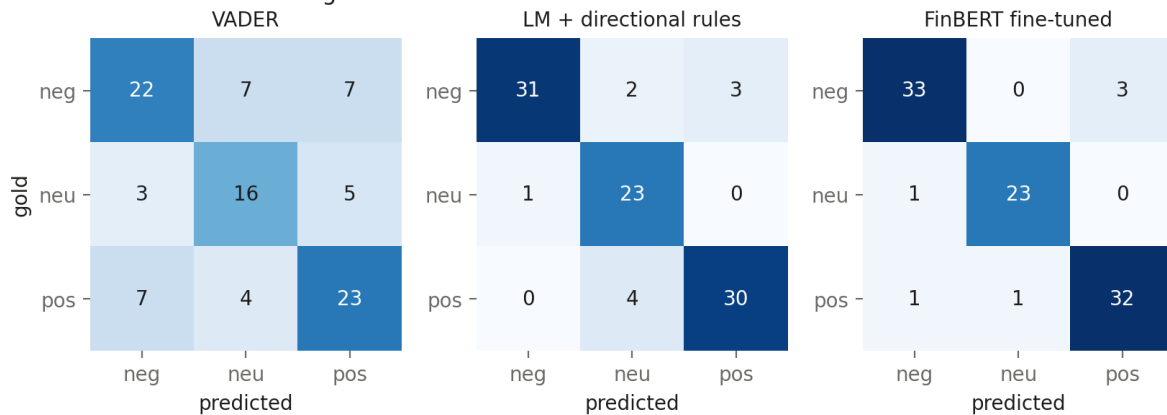


Figure 6.2: Confusion matrices for the general-purpose lexicon, the best rule model and the best learned model.

VADER's errors are spread across all cells, including 14 negative/positive swaps — the direction errors that matter most for an investor. FinBERT's six errors are mostly positive/negative confusions on trap

sentences about costs and cash (Section 6.3); the rule model's errors are mostly confusions with neutral.

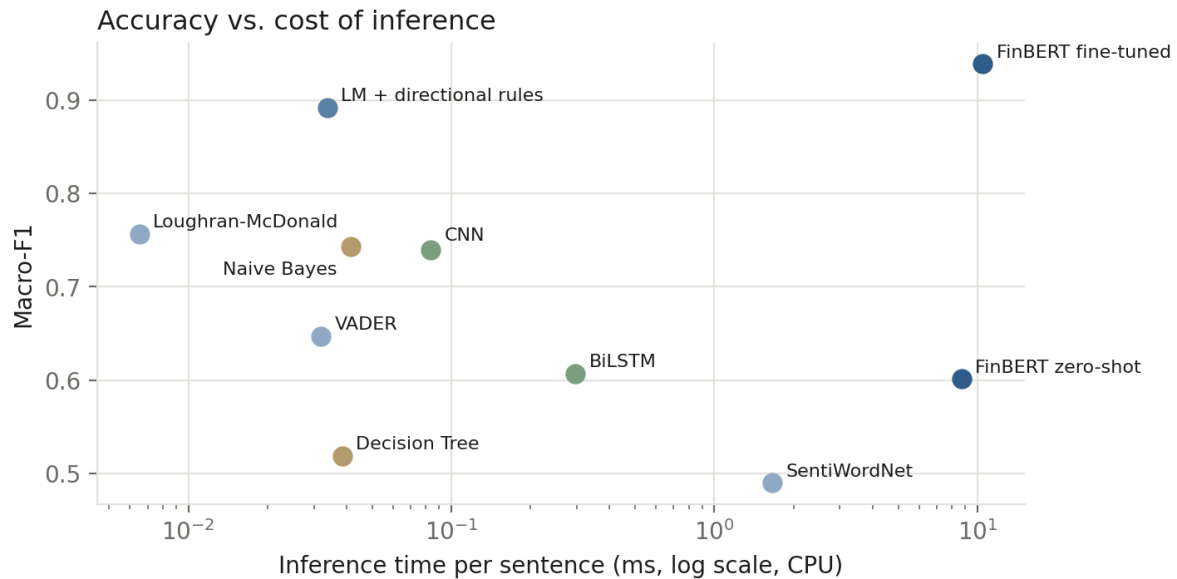


Figure 6.3: Accuracy against inference cost per sentence on CPU (log scale).

The rule model scores a sentence in about 0.03 ms; FinBERT takes about 10 ms, three hundred times longer. At watchlist scale both are affordable, which is why the registry serves FinBERT, but the rule model is a strong fallback and remains the default when no trained weights are present.

Why these numbers are optimistic

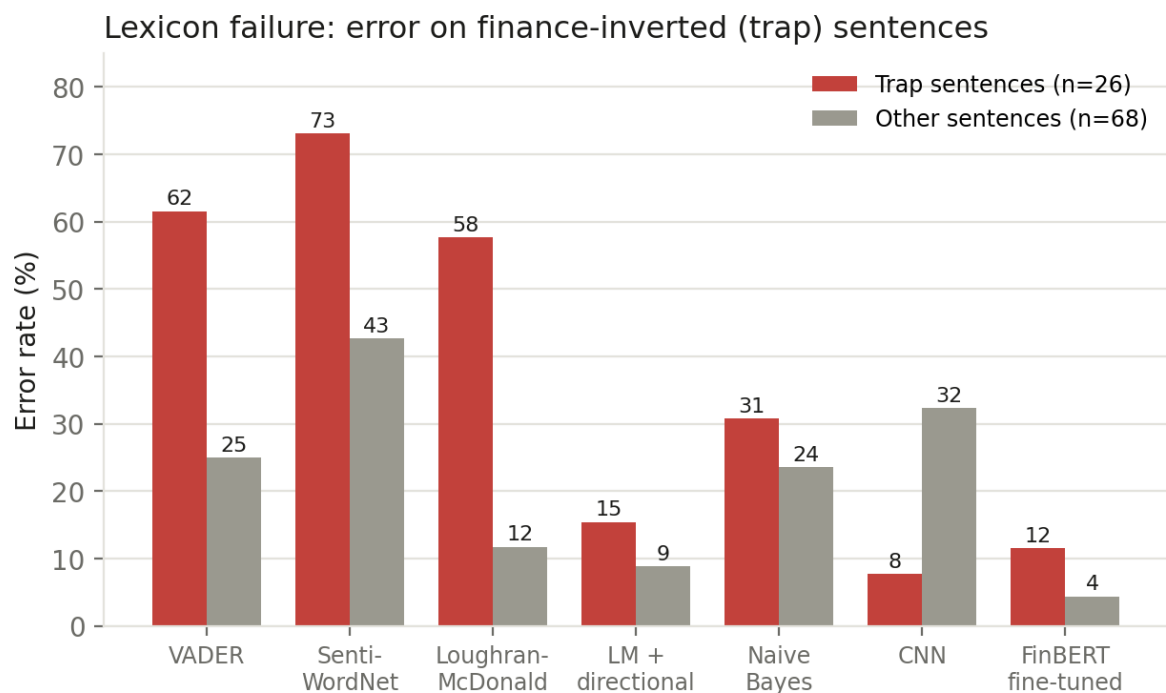
- **Weak labels are too easy.** CNN, BiLSTM and FinBERT all reached 100% accuracy on the weak-label validation split within a few epochs, yet 74%–94% on gold. The templates are far more regular than real language; validation accuracy on them says nothing about real performance.
- **The gold set is small.** One sentence is about 1.1 accuracy points; differences of a few points between models are within noise, and only one seed was run.
- **Gold and templates share phrasing.** Both were written by the same author about the same six aspects, so the trained models' gold scores will overstate performance on real transcripts.
- **The rule model was written with sight of the gold set.** Its directional rules reflect patterns the author had read; its score is an optimistic upper bound. (Gold-derived words were removed from the plain LM seed list to keep that baseline fair.)
- **The LM lexicon is a curated subset,** not the official ~86,000-word dictionary.

6.3 Lexicon failure analysis

Table 6.3 and Figure 6.4 split each model's error rate between the 26 trap sentences and the other 68.

Table 6.3: Error rate on trap and non-trap sentences

Model	All (n=94)	Trap (n=26)	Non-trap (n=68)
VADER	35.1%	61.5%	25.0%
SentiWordNet	51.1%	73.1%	42.6%
Loughran-McDonald (seed list)	24.5%	57.7%	11.8%
LM + directional rules	10.6%	15.4%	8.8%
Naive Bayes (TF-IDF)	25.5%	30.8%	23.5%
CNN (Kim-style)	25.5%	7.7%	32.4%
FinBERT, fine-tuned	6.4%	11.5%	4.4%

**Figure 6.4:** Error rate on finance-inverted (trap) sentences versus the rest.

General-purpose lexicons are wrong on most trap sentences: 61.5% for VADER and 73.1% for SentiWordNet. Plain Loughran-McDonald counting fixes much of the vocabulary problem on ordinary sentences (11.8% error) but not the trap problem (57.7%), because a word list cannot tell whether *decreased* is good or bad without knowing what decreased. Directional rules (15.4%) and the learned models close most of the gap. Table 6.4 lists representative failures.

Table 6.4: Representative trap sentences mis-scored by VADER

Aspect	Sentence	Gold	VADER	LM+dir.	FinBERT
demand	Customer churn decreased to its lowest level in three years.	pos	neg	✓	✓
liquidity	Total liabilities decreased and our net debt fell to its lowest level in a decade.	pos	neg	✓	✓
litigation	Legal expenses decreased as the dispute was resolved earlier than planned.	pos	neg	✓	✓

Aspect	Sentence	Gold	VADER	LM+dir.	FinBERT
litigation	Litigation expenses increased this quarter as the patent case moved toward trial.	neg	pos	✓	✓
litigation	The jury verdict went against us and we recorded a 90 million dollar provision.	neg	pos	✓	✓
demand	Churn increased in the mid-market as customers consolidated vendors.	neg	pos	✓	✓
margins	Cost of revenue increased 14% year over year, outpacing sales growth.	neg	pos	✓	✗ pos
liquidity	Cash declined by 400 million dollars and we fully drew our credit facility.	neg	pos	✓	✗ pos
guidance	We now expect operating expenses to come in below the low end of our prior guidance.	pos	neg	✓	✗ neg
guidance	We do not expect to meet our prior guidance for free cash flow this year.	neg	pos	✗	✓

The residual FinBERT errors are the same phenomenon in a subtler form: *increased* next to a cost, or *declined* next to cash, pulls the model toward the polarity of the verb rather than the implication for the company. This supports the project's premise that financial directionality must be modelled explicitly or learned from data that contains it.

```

lexicon failure analysis

PS Sentari> python -m evaluation.lexicon_failure
| model | error (all) | error (trap) | error (non-trap) |
|---|---|---|---|
| vader | 35.1% | 61.5% | 25.0% |
| sentiwordnet | 51.1% | 73.1% | 42.6% |
| lm_lexicon | 24.5% | 57.7% | 11.8% |
| lm_directional | 10.6% | 15.4% | 8.8% |
| nb_tfidf | 25.5% | 30.8% | 23.5% |
| cnn | 25.5% | 7.7% | 32.4% |
| finbert_ft | 6.4% | 11.5% | 4.4% |

```

Figure 6.5: Output of `python -m evaluation.lexicon_failure`.

6.4 Grounding audit

Adversarial audit. Starting from 43 faithful claims (verbatim scored sentences), the audit creates five kinds of corrupted claims that imitate how a generator fails, passes all of them through the Skeptic, and counts a corrupted claim as *caught* if it is rejected or flagged unverified — both keep it out of the brief body.

Table 6.5: Adversarial grounding audit (caught = rejected or flagged; hard reject = rejected)

Corruption	n	Heuristic caught	Heuristic rejected	NLI caught	NLI rejected
Number changed	13	100.0%	100.0%	100.0%	100.0%
Citation removed	43	100.0%	100.0%	100.0%	100.0%
Wrong chunk cited	43	100.0%	51.2%	100.0%	34.9%
Fabricated claim	43	100.0%	32.6%	100.0%	58.1%
Polarity flipped	22	22.7%	22.7%	100.0%	100.0%
Faithful claims (false alarms)	43	0 flagged	0 rejected	0 flagged	0 rejected

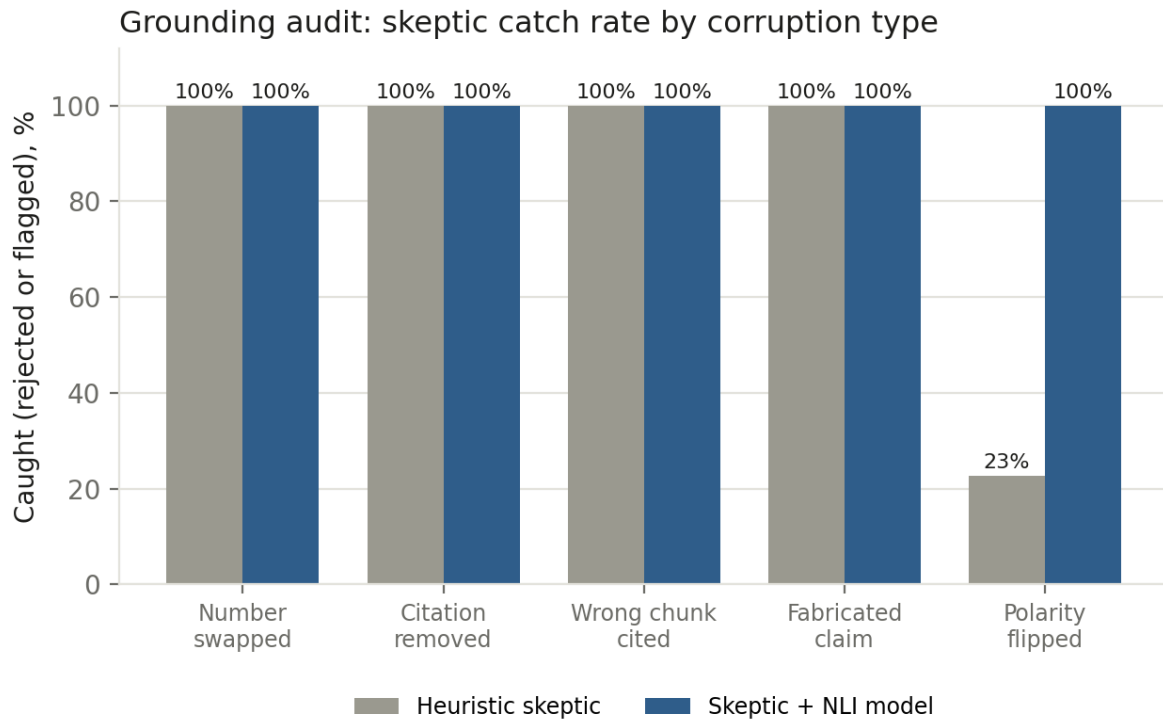


Figure 6.6: Skeptic catch rate by corruption type, heuristic versus NLI entailment.

Structural failures — invented numbers, missing or wrong citations, fabricated claims — are caught in both modes, and no faithful claim was wrongly flagged. The heuristic Skeptic, however, let 17 of 22 polarity-flipped claims through ("*gross margin contracted 180 basis points*" citing a sentence that says it *expanded*): the claim shares almost every word and number with its source, so similarity checks pass, and the polarity-cue heuristic is too coarse. With the NLI model all 22 were rejected. The NLI backend should therefore be treated as required, not optional, for real use.

Caveats. The corruptions are synthetic and the faithful claims are verbatim, which is the easy case; paraphrased LLM claims are harder to verify. The audit therefore bounds the Skeptic's catch rate from above. A human audit — `grounding_audit --export N` samples claims from real briefs into a spreadsheet for 1/0 annotation and `--score` computes the supported fraction and the Skeptic's agreement with the annotator — is implemented and tested but has not been run, because no LLM-written briefs have been generated yet.

6.5 Signal validity

The question is whether the prepared-versus-Q&A delta correlates with the subsequent five-day return (market-adjusted) better than document-level sentiment does. The harness builds three predictors per call (delta, document-level mean, prepared-only mean), joins them to forward returns from a CSV or from Yahoo Finance, and reports Spearman ρ with p-value, Pearson r , a 2,000-sample bootstrap 95% confidence interval, and the difference between the delta and document-level correlations, together with an explicit interpretation that defaults to a null result when nothing is significant.

Not run. The only corpus available is synthetic, so any correlation with real prices would be meaningless, and no returns file is bundled on purpose. Tests exercise the harness on fake returns only to check the plumbing. With a watchlist-sized sample (tens of calls) a null result is the expected outcome; the project plan treats a clearly reasoned null result as a legitimate finding.

6.6 Latency and cost

Table 6.6: Measured latency

Operation	Measured	Notes
Score one sentence	0.03 ms (rules) – 10.5 ms (FinBERT)	CPU, gold-set average (Table 6.2)
Generate one brief, offline agents	0.79 s mean, 1.55 s max	Rule-based model, heuristic Skeptic, 2 briefs
Generate one brief, NLI Skeptic	0.4 – 2.5 s	NLI model already loaded (NVLT, ACMX)
Load the NLI model	4 s – 51 s	One-off per process; varies with disk cache and Hugging Face hub checks
Full daily run on the sample corpus	84 s	Fresh database; 5 documents, 88 chunks, FinBERT scoring, NLI Skeptic, 2 briefs, drift, digest; includes loading both models
LLM tokens and cost per brief	not measured	Recorded per brief (tokens in/out) once an API key is configured
Fine-tuning time	CNN 10 s · BiLSTM 28 s · FinBERT 238 s	CPU; FinBERT on 1,500 examples, 2 epochs

Cached briefs cost nothing: a second daily run with no new documents skipped all five documents as duplicates, wrote no new scores or embeddings and returned both cached briefs. It did re-analyse six chunks that contain no aspect mention (they leave no score row to mark them as done) — harmless, but wasted work at scale.

6.7 Qualitative findings from the end-to-end run

- **Q&A is weaker than the script.** On ACMX's first call the Q&A-minus-prepared delta was -1.15 , driven by guidance (-1.99 : "raising our full-year guidance" in the script, "visibility beyond the third quarter is limited" in the Q&A) and margins (-1.98 : a 180 bp expansion in the script, input costs "a headwind" and tariffs "difficult to predict" in the Q&A). The signal behaves as designed on text written to exhibit it; whether it does so on real calls is the open question of Section 6.5.
- **Repeated claims** across aspects in the first version of the brief, now fixed (Section 5.3).
- **Over-confident transformer.** FinBERT assigned -1.00 with 100% confidence to "*We are not able to say.*" — a reasonable negative-tone reading, but the certainty is not warranted; confidence values are uncalibrated.

- **Mixed document types.** The top digest move (ACMX demand $+0.15 \rightarrow +0.99$) compares a call with a 10-Q filed six days later; trajectories mix document types, which inflates moves.
- **Noisy drift alerts** on very small windows (Section 5.4).

7 Testing

7.1 Strategy

Every test runs offline in about 30 seconds with no API keys: the LLM backend is pinned to the deterministic heuristic, the serving model to the rule-based model, and the `.env` loader is disabled, so results never depend on a developer's local files. Tests use in-memory SQLite, the bundled sample corpus and FastAPI's test client.

Table 7.1: Automated tests by module (58 in total; one test is parametrised six ways)

Module	Tests	What is covered
<code>test_ingestion.py</code>	6	normalisation, abbreviation-aware splitting, speaker/section parsing, hashing, idempotent ingestion
<code>test_absa.py</code>	20	negation scope, hedging, clause splitting, aspect triggers, spaCy vs. fallback agreement, directional scoring, VADER's failure, gold-set integrity, no gold leakage into training data, trainable model beats chance
<code>test_agents.py</code>	12	Skeptic accepts faithful claims; rejects fabricated numbers, bad citations and contradictions; full graph produces a cited brief with every agent in the trace; brief caching; rejected claims never reach the brief; no sentence cited twice by one side; Judge merges duplicate claims
<code>test_api_and_monitoring.py</code>	9	PSI, drift alerting and idempotency, API validation and errors, daily run followed by every dashboard endpoint, cron-token enforcement
<code>test_evaluation.py</code>	6	lexicon failure analysis, adversarial grounding thresholds, human-audit export/score round trip, signal-validity plumbing and null reporting, latency report
<code>test_env_loader.py</code>	5	real environment wins over <code>.env</code> , empty values ignored, skip flag, parser edge cases, every variable documented

```

pytest
PS Sentari> python -m pytest -q
..... [100%]
58 passed in 29.44s

```

Figure 7.1: Test run: 58 passed.

7.2 Continuous integration

A GitHub Actions workflow installs CPU-only PyTorch and the NLP models, runs the Python suite, and type-checks and builds the dashboard on every push and pull request. (Its results on GitHub were not inspected during this phase.)

7.3 Defects found and fixed

Table 7.2: Defects found through testing and use

Defect	Found by	Fix
Bull and bear claim IDs both started with <i>b</i> , so a rejected bear claim could hide an accepted bull claim	Test: rejected claims never reach the brief	Distinct b1 / br prefixes
Skeptic rejected a verbatim quote containing negation as "contradicted"	Inspecting a generated brief	Near-verbatim quotes exempt from the cue heuristic
Offline LLM backend reported its name as <i>base</i> , breaking cost reports	Test: latency/cost report	Name set in the constructor
Dashboard shipped a Next.js release with known security advisories	npm audit	Upgraded to Next.js 16 / React 19 (0 advisories)
Daily CLI crashed printing the digest (▲ ▼ △) on Windows consoles	Running the pipeline for this report	stdout reconfigured to UTF-8
Each daily run appended duplicate drift reports	Screenshot of the drift page for this report	One report per (ticker, window); regression test added
Briefs repeated one sentence as several claims (one per aspect it triggered)	Screenshot of the brief for this report	Each sentence cited once per side; Judge merges duplicate claims; 3 tests added
Navigation bar overflowed the screen at phone width	Phone-width screenshot for this report	Nav wraps; disclaimer on its own line below 600 px

8 Syllabus Mapping

Table 8.1: Mapping to the Sentiment Analysis course (231CAUEC51)

Module	Topic	Where it appears in Sentari
M1	Levels, challenges, applications	Aspect-level framing (1.1, 2.1); non-advice ethics (9)
M2	Preprocessing and feature engineering	Transcript-aware segmentation, negation scope, hedging, clause splitting, TF-IDF, embeddings (4.1, 4.2)
M3	Lexicon- and knowledge-based methods	VADER, SentiWordNet, Loughran-McDonald, directional rules; lexicon failure analysis (4.4, 6.3)
M4	Machine-learning and deep-learning methods	Naive Bayes, Decision Tree, CNN, BiLSTM, fine-tuned FinBERT — one ladder, one test set (4.4, 6.2)
M5	Aspect-based sentiment analysis	Aspect triggers with dependency-parsed opinion targets, aspect-pair transformer, per-aspect trajectories (4.3–4.5)
M6	Applications	Financial decision support: dashboard, digest, grounded briefs, drift monitoring (4.7–4.10, 5)

9 Ethical and Legal Considerations

- **Not investment advice.** Sentari describes what companies said; it makes no recommendation and no price prediction. The disclaimer appears in the API description and every response that carries analysis, on every dashboard page, and in every digest and brief.
- **Honest uncertainty.** Confidence is labelled a heuristic, unverified claims are visibly flagged, and the evaluation reports negative and missing results alongside positive ones.
- **Source terms.** Official APIs are preferred (SEC EDGAR with a contact User-Agent, as the SEC requires). Scrapers respect terms of use; the NSE scraper returns nothing rather than work around access controls. Raw responses are cached to minimise load on sources.
- **Data protection.** Only public corporate disclosures and news are processed; no personal data is collected. Speaker names in transcripts are public officers' statements. API keys live in a gitignored `.env` and never in code or logs.
- **Synthetic data.** The demonstration corpus is fictional and labelled SYNTHETIC in every document title shown in the UI.

10 Project Status Against the Plan

Table 10.1: Roadmap from the project specification

Phase	Deliverable	Status
1 Data pipeline	Ingestion for a 10-ticker pilot watchlist	Pipeline built and tested on sample data; live pilot pending a transcript source and watchlist
2 Preprocessing & lexicons	Clean corpus; VADER/SWN/LM scored; failure analysis	Done
3 Classical ML + DL	NB, DT, CNN, LSTM trained and logged in MLflow	Done
4 Aspect extraction	POS/dependency-based extraction; aspect-level scoring	Done (rules + dependency parse; supervised aspect tagger not built)
5 Transformer	FinBERT fine-tuned on aspect sentiment	Done (on weak labels)
6 Agent layer	Bull/bear/skeptic/judge with grounding checks	Done
7 Storage + dashboard	Schema live; dashboard with drill-down	Done (SQLite verified; PostgreSQL untested)
8 Deployment + monitoring	Dockerised, on Cloud Run, drift monitor running	Drift monitor done; Docker/Cloud Run files written, not built or deployed
9 Evaluation	Ladder table, grounding audit, signal validity	Ladder and adversarial audit done; human audit and signal validity pending real data
10 Report + demo	Final report, recorded demo, GitHub polish	This report; demo video not recorded

10.1 Remaining work, in priority order

1. **Real transcripts.** Choose a source (public dataset, NSE/BSE transcript PDFs, company IR pages or a paid API), write the scraper, and ingest the 10-ticker pilot.
2. **Real gold set.** Annotate a few hundred sentences from real filings and calls, then re-run the ladder and the lexicon analysis against the official Loughran-McDonald dictionary.
3. **Signal validity** on the pilot with real forward returns; report the result whatever it is.
4. **LLM agents** with an API key; human grounding audit of about 50 claims; measure tokens and cost per brief.
5. **Deploy:** build the images, run on Cloud Run with Cloud SQL and the scheduler, then record the demo.
6. **Remaining fixes found in this report:** raise the drift minimum sample; separate trajectories by document type; re-analysing aspect-less chunks on every run.

11 Limitations and Future Work

11.1 Limitations

- **Evaluation data.** A 94-sentence self-written gold set, weak template labels and a synthetic corpus limit what the numbers can show; Section 6.2 lists the specific biases.
- **Aspect handling.** One sentence can still be *tagged* with several aspects (it is now cited only once per side); mixed clauses of one aspect are averaged, so a strong positive and a strong negative cancel to neutral; aspect detection is rule-based with no learned tagger, so a word like *outlook* can file a tone statement under guidance.
- **Grounding.** The heuristic Skeptic cannot detect polarity inversions; even with NLI, verification is only as good as the entailment model, and has not been tested on paraphrased LLM claims.
- **Calibration.** Model confidences (notably FinBERT's) and the brief confidence are not calibrated probabilities.
- **Monitoring.** Drift alerts on very small windows are noisy; trajectories mix document types.
- **Deployment.** Live scrapers, containers, PostgreSQL, cloud deployment and digest sending are implemented but unverified.

11.2 Future work

- A supervised aspect tagger trained on the real gold set, to reduce multi-aspect and mis-filed sentences.
- Calibrated confidence (temperature scaling on a real validation set) and per-aspect thresholds.
- Multiple training seeds and bootstrap confidence intervals for every ladder metric.
- pgvector or Qdrant approximate-nearest-neighbour retrieval over sentence-transformer embeddings for larger watchlists.
- Speaker-level analysis (CEO versus CFO tone) and question-level Q&A analysis (which analyst question triggered hedging).
- Indian-market coverage through exchange-filed transcript PDFs, including PDF text extraction.

12 Conclusion

Sentari shows that aspect-level analysis of financial disclosures can be delivered as a working, end-to-end service: from raw documents, through six-aspect sentiment scored by a comparable ladder of ten models, to a written brief in which every claim is tied to its source sentence and checked by an explicit grounding gate before a reader sees it.

The measurements support the project's two central premises. General-purpose lexicons fail on financial language in a specific, predictable way — they read the direction of a verb rather than its implication for the company — and both finance-specific directionality and fine-tuned transformers largely correct it. And a grounding gate does stop the structural errors that generative models make; the adversarial audit also exposed where a heuristic gate fails and showed that an entailment model closes that gap.

Equally, the report is explicit about what is not yet shown. The model numbers come from a small self-written test set and weak labels; the corpus is synthetic; and the platform's signature signal has not been tested against real prices. The next steps are therefore clear and practical: real transcripts, a larger real gold set, the signal-validity study and a deployed service. The pipeline, evaluation harnesses and tests needed for each of them are already in place.

References

- [1] T. Loughran and B. McDonald, "When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks," *The Journal of Finance*, vol. 66, no. 1, pp. 35–65, 2011.
- [2] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. 8th Int. AAAI Conf. on Weblogs and Social Media (ICWSM)*, 2014.
- [3] S. Baccianella, A. Esuli and F. Sebastiani, "SentiWordNet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining," in *Proc. LREC*, 2010.
- [4] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," arXiv:1908.10063, 2019.
- [5] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. EMNLP*, 2014, pp. 1746–1751.
- [6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.
- [8] C. Sun, L. Huang and X. Qiu, "Utilizing BERT for aspect-based sentiment analysis via constructing auxiliary sentence," in *Proc. NAACL-HLT*, 2019.
- [9] M. Pontiki et al., "SemEval-2014 Task 4: Aspect based sentiment analysis," in *Proc. 8th Int. Workshop on Semantic Evaluation (SemEval)*, 2014.
- [10] B. Liu, *Sentiment Analysis and Opinion Mining*. Morgan & Claypool, 2012.
- [11] P. Malo, A. Sinha, P. Korhonen, J. Wallenius and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," *JASIST*, vol. 65, no. 4, pp. 782–796, 2014.
- [12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019. (Cross-encoder NLI model `cross-encoder/nli-deberta-v3-small` from the Sentence-Transformers library.)
- [13] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.
- [14] N. Siddiqi, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Wiley, 2006.
- [15] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [16] U.S. Securities and Exchange Commission, "EDGAR application programming interfaces," [sec.gov developer resources](https://www.sec.gov/developer).
- [17] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum, 1988.

- [18] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.
- [19] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020.
- [20] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, 2018.

Appendix A Running the System

Prerequisites: Python 3.12+, Node.js 20+. All commands run from the repository root.

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm

# tests (offline) and the daily pipeline
python -m pytest -q
python -m absa_service.cli daily
python -m absa_service.cli analyze "<any earnings text>"

# model ladder and evaluation
python -m absa_service.train --transformer --transformer-zero-shot
python -m evaluation.lexicon_failure
SENTARI_NLI=hf python -m evaluation.grounding_audit --adversarial
python -m evaluation.signal_validity --returns returns.csv
python -m evaluation.report

# services
uvicorn absa_service.main:app --port 8000      # API; docs at /docs
cd dashboard && npm install && npm run dev     # http://localhost:3000
docker compose up --build                     # full stack (not yet verified)
```

Appendix B Configuration

Copy `.env.example` to `.env` (gitignored). Everything is optional for offline use; `SETUP.md` explains how to obtain each value.

Table B.1: Configuration variables

Variable	Purpose
DATABASE_URL	SQLAlchemy URL; default local SQLite
SENTARI_MODEL	Force the serving model (else the registry decides)
ANTHROPIC_API_KEY, SENTARI_LLM, SENTARI_LLM_MODEL	LLM agents; SENTARI_LLM=heuristic forces offline
SENTARI_NLI=hf	Transformer NLI entailment in the Skeptic (recommended)
SENTARI_USER_AGENT	Contact User-Agent required by SEC EDGAR
SENTARI_EMBEDDER, SENTARI_GLOVE, SENTARI_BASE_MODEL	Embeddings, GloVe vectors, transformer checkpoint
MLFLOW_TRACKING_URI	MLflow server (default local file store)
SLACK_WEBHOOK_URL, SMTP_*, DIGEST_TO	Digest delivery
SENTARI_CRON_TOKEN, CORS_ORIGINS	Protect <code>/run/daily</code> ; allowed dashboard origins
SENTARI_ARTIFACTS, SENTARI_CACHE_DIR	Model weights and raw-response cache locations

Appendix C API Reference

Table C.1: HTTP endpoints

Method	Path	Purpose
GET	/health	Service status, aspect list, disclaimer
POST	/analyze	Aspect sentiment for free text
POST	/score	Raw polarity for (text, aspect) pairs with any model
GET	/models	Model registry: runs, metrics, serving flag
GET	/watchlist	Per-ticker latest aspect summary and brief
GET	/tickers/{ticker}/trajectory	Per-document aspect means over time
GET	/tickers/{ticker}/points	Scored sentences behind a trajectory point
GET	/chunks/{chunk_id}	Source sentence with context and all model verdicts
GET	/tickers/{ticker}/briefs	Recent briefs for a ticker
GET	/briefs/{brief_id}	Brief with audit trail and resolved sources
POST	/briefs/generate	Generate or fetch the cached brief
GET	/drift	Drift reports
GET	/digest	Current top aspect moves
POST	/run/daily	Daily pipeline (scheduler target; optional token)

Appendix D Repository Structure

Table D.1: Repository layout

Path	Contents
ingestion/	scrapers/ (sample, sec_edgar, news, nse), normalize.py, pipeline.py, scheduler_config.yaml
absa_service/	preprocessing/, models/ (lexicon, classical_ml, deep_learning, transformer), aspect_extraction.py, scoring.py, signals.py, train.py, registry.py, serving.py, main.py + api.py (FastAPI), orchestration.py, cli.py
agents/	extractor_agent.py, bull_bear_agents.py, skeptic_agent.py, judge_agent.py, graph.py (LangGraph), llm.py
storage/	models.py (schema), db.py, migrations/
monitoring/, delivery/	drift.py; digest.py
dashboard/	Next.js app: watchlist, ticker, brief, drift and model pages
evaluation/	metrics, lexicon_failure, grounding_audit, signal_validity, latency_cost, report, model_comparison.ipynb
data/	sample/ (synthetic corpus), gold/gold.tsv (evaluation set)
results/	committed evaluation outputs quoted in this report
report/	this report, its figures and screenshots, and the scripts that regenerate them
tests/	58 automated tests
root	Dockerfile, docker-compose.yml, dvc.yaml, .github/workflows/ci.yml, deploy/, .env.example, SETUP.md, CLAUDE.md

Development tooling. Git and GitHub for version control; pytest; Playwright for the screenshots in this report; and Claude Code, an AI coding assistant, used during development.